

CHAPTER 10

Sorting Algorithms

IN THIS CHAPTER, YOU WILL:

- Learn the various sorting algorithms
- Explore how to implement selection sort, insertion sort, Shellsort, quicksort, mergesort, and heapsort
- Discover how the sorting algorithms discussed in this chapter perform
- Learn how priority queues are implemented

Chapter 9 discussed the search algorithms on lists. A sequential search does not assume that the data is in any particular order; however, as noted, this search does not work efficiently for large lists. By contrast, a binary search is very fast for array-based lists, but it requires the data to be in order. Because a binary search requires the data to be in order and its performance is good for array-based lists, this chapter focuses on sorting algorithms.

Sorting Algorithms

There are several sorting algorithms in the literature. In this chapter, we discuss some of the most commonly used sorting algorithms. To compare the performance of these algorithms, we also provide the analysis of these algorithms. These sorting algorithms can be applied to either array-based lists or linked lists. We will specify whether the algorithm being developed is for array-based lists or linked lists.

The functions implementing these sorting algorithms are included as `public` members of the related class. (For example, for an array-based list, these are the members of the `class arrayListType`.) By doing so, the algorithms have direct access to the list elements.

Suppose that the sorting algorithm selection sort (described in the next section) is to be applied to array-based lists. The following statements show how to include selection sort as a member of the `class arrayListType`:

```
template <class elemType>
class arrayListType
{
public:
    void selectionSort();
    ...
};
```

Selection Sort: Array-Based Lists

In selection sort, a list is sorted by selecting elements in the list, one at a time, and moving them to their proper positions. This algorithm finds the location of the smallest element in the unsorted portion of the list and moves it to the top of the unsorted portion (that is, the whole list) of the list. The first time we locate the smallest item in the entire list, the second time we locate the smallest item in the list starting from the second element in the list, and so on. Selection sort described here is designed for array-based lists.

Suppose you have the list shown in Figure 10-1.

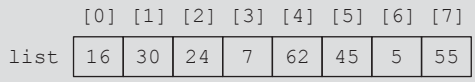


FIGURE 10-1 List of 8 elements

Figure 10-2 shows the elements of `list` in the first iteration.

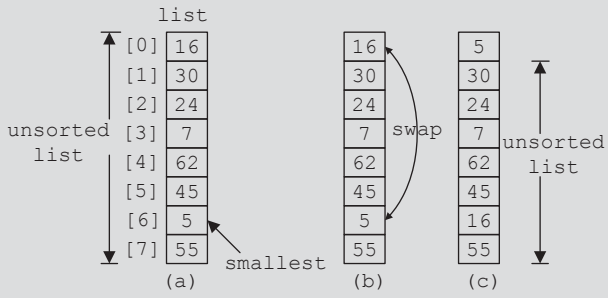


FIGURE 10-2 Elements of `list` during the first iteration

Initially, the entire list is unsorted. So we find the smallest item in the list. The smallest item is at position 6, as shown in Figure 10-2(a). Because this is the smallest item, it must be moved to position `0`. So we swap `16` (that is, `list[0]`) with `5` (that is, `list[6]`), as shown in Figure 10-2(b). After swapping these elements, the resulting list is as shown in Figure 10-2(c).

Figure 10-3 shows the elements of `list` in the second iteration.

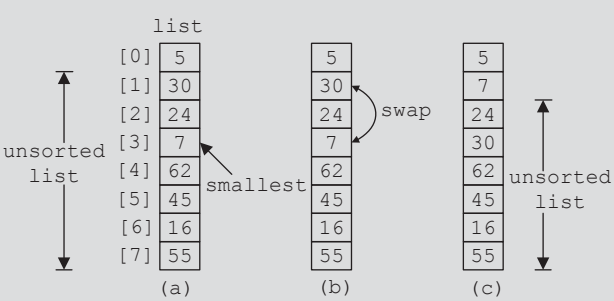


FIGURE 10-3 Elements of `list` during the second iteration

Now the unsorted list is `list[1]...list[7]`. So we find the smallest element in the unsorted list. The smallest element is at position 3, as shown in Figure 10-3(a). Because the smallest element in the unsorted list is at position 3, it must be moved to position 1. So we swap `7` (that is, `list[3]`) with `30` (that is, `list[1]`), as shown in Figure 10-3(b). After swapping `list[1]` with `list[3]`, the resulting list is as shown in Figure 10-3(c).

Now the unsorted list is `list[2]...list[7]`. So we repeat the preceding process of finding the (position of the) smallest element in the unsorted portion of the list and moving it to the beginning of the unsorted portion of the list. Selection sort, thus, involves the following steps.

In the unsorted portion of the list:

1. Find the location of the smallest element.
2. Move the smallest element to the beginning of the unsorted list.

Initially, the entire list, `list[0]...list[length-1]`, is the unsorted list. After executing Steps 1 and 2 once, the unsorted list is `list[1]...list[length-1]`. After executing Steps 1 and 2 a second time, the unsorted list is `list[2]...list[length-1]`, and so on. We can keep track of the unsorted portion of the list and repeat Steps a and b with the help of a `for` loop as follows:

```
for (index = 0; index < length - 1; index++)
{
   1. Find the location, smallestIndex, of the smallest element in
      list[index]...list[length - 1].
   2. Swap the smallest element with list[index]. That is, swap
      list[smallestIndex] with list[index].
}
```

The first time through the loop, we locate the smallest element in `list[0]...list[length-1]` and swap this smallest element with `list[0]`. The second time through the loop, we locate the smallest element in `list[1]...list[length-1]` and swap this smallest element with `list[1]`, and so on. This process continues until the length of the unsorted list is `1`. (Note that a list of length `1` is sorted.) It, therefore, follows that to implement selection sort, we need to implement Steps 1 and 2.

Given the starting index, `first`, and the ending index, `last`, of the list, the following C++ function returns the index of the smallest element in `list[first]...list[last]`:

```
template <class elemType>
int arrayListType<elemType>::minLocation(int first, int last)
{
    int minIndex;

    minIndex = first;
```

```
    for (int loc = first + 1; loc <= last; loc++)
        if( list[loc] < list[minIndex])
            minIndex = loc;

    return minIndex;
} //end minLocation
```

Given the locations in the list of the elements to be swapped, the following C++ function, `swap`, swaps those elements:

```
template <class elemType>
void arrayListType<elemType>::swap(int first, int second)
{
    elemType temp;

    temp = list[first];
    list[first] = list[second];
    list[second] = temp;
}//end swap
```

We can now complete the definition of the function `selectionSort`:

```
template <class elemType>
void arrayListType<elemType>::selectionSort()
{
    int minIndex;

    for (int loc = 0; loc < length - 1; loc++)
    {
        minIndex = minLocation(loc, length - 1);
        swap(loc, minIndex);
    }
}
```

You can add the functions to implement selection sort in the definition of the `class arrayListType` as follows:

```
template<class elemType>
class arrayListType
{
public:
     //Place the definitions of the function given earlier here.

    void selectionSort();
    ...

private:
      //Place the definitions of the members given earlier here.
    void swap(int first, int second);
    int minLocation(int first, int last);
};
```

EXAMPLE 10-1

The following program tests selection sort:

```
//*************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use selection sort in a
// program.
//*************************************************************

#include <iostream>                                          //Line 1
#include "arrayListType.h"                                   //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    arrayListType<int> list;                                 //Line 6
    int num;                                                 //Line 7

    cout << "Line 8: Enter numbers ending with -999"
         << endl;                                            //Line 8

    cin >> num;                                              //Line 9

    while (num != -999)                                      //Line 10
    {                                                        //Line 11
        list.insert(num);                                    //Line 12
        cin >> num;                                          //Line 13
    }                                                        //Line 14

    cout << "Line 15: The list before sorting:" << endl;     //Line 15
    list.print();                                            //Line 16
    cout << endl;                                            //Line 17

    list.selectionSort();                                    //Line 18

    cout << "Line 19: The list after sorting:" << endl;      //Line 19
    list.print();                                            //Line 20
    cout << endl;                                            //Line 21

    return 0;                                                //Line 22
}                                                            //Line 23
```

Sample Run: In this sample run, the user input is shaded.

```
Line 8: Enter numbers ending with -999
34 67 23 12 78 56 36 79 5 32 66 -999
Line 15: The list before sorting:
34 67 23 12 78 56 36 79 5 32 66
```

```
Line 19: The list after sorting:
5 12 23 32 34 36 56 66 67 78 79
```

For the most part, the preceding output is self-explanatory. Notice that the statement in Line 12 calls the function `insert` of the `class arrayListType`. Similarly, the statements in Lines 16 and 20 call the function `print` of the `class arrayListType`. The statement in Line 18 calls the function `selectionSort` to sort the list.

NOTE

1. Selection sort can also be implemented by selecting the largest element in the (unsorted portion of the) list and moving it to the bottom of the list. You can easily implement this form of selection sort by altering the `if` statement in the function `minLocation`, and passing the appropriate parameters to the corresponding function and the function `swap`, when these functions are called in the function `selectionSort`.
2. Selection sort can also be applied to linked lists. The general algorithm is the same, and the details are left as an exercise for you. See Programming Exercise 1 at the end of this chapter.

Analysis: Selection Sort

In the case of search algorithms (Chapter 9), our only concern was with the number of key (item) comparisons. A sorting algorithm makes key comparisons and also moves the data. Therefore, in analyzing the sorting algorithm, we look at the number of key comparisons as well as the number of data movements. Let us look at the performance of selection sort.

Suppose that the length of the list is n. The function `swap` does three item assignments and is executed $n - 1$ times. Hence, the number of item assignments is $3(n - 1)$.

The key comparisons are made by the function `minLocation`. For a list of length k, the function `minLocation` makes $k - 1$ key comparisons. Also, the function `minLocation` is executed $n - 1$ times (by the function `selectionSort`). The first time, the function `minLocation` finds the index of the smallest key item in the entire list and so makes $n - 1$ comparisons. The second time, the function `minLocation` finds the index of the smallest element in the sublist of length $n - 1$ and so makes $n - 2$ comparisons, and so on. Hence the number of key comparisons is as follows:

$$(n-1) + (n-2) + \cdots + 2 + 1 = (1/2)n(n-1) = (1/2)n^2 - (1/2)n = O(n^2)$$

Thus, it follows that if $n = 1000$, the number of key comparisons the selection sort makes is $1/2(1000^2) - 1/2(1000) = 499500 \approx 500000$.

Insertion Sort: Array-Based Lists

The previous section described and analyzed the selection sort algorithm. It was shown that if $n = 1000$, the number of key comparisons is approximately 500,000, which is quite high. This section describes the sorting algorithm called insertion sort, which tries to improve—that is, reduce—the number of key comparisons.

Insertion sort sorts the list by moving each element to its proper place. Consider the list given in Figure 10-4.

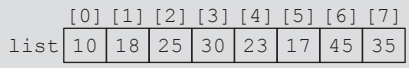


FIGURE 10-4 `list`

The length of the list is `8`. In this list, the elements `list[0]`, `list[1]`, `list[2]`, and `list[3]` are in order. That is, `list[0]...list[3]` is sorted, as shown in Figure 10-5(a).

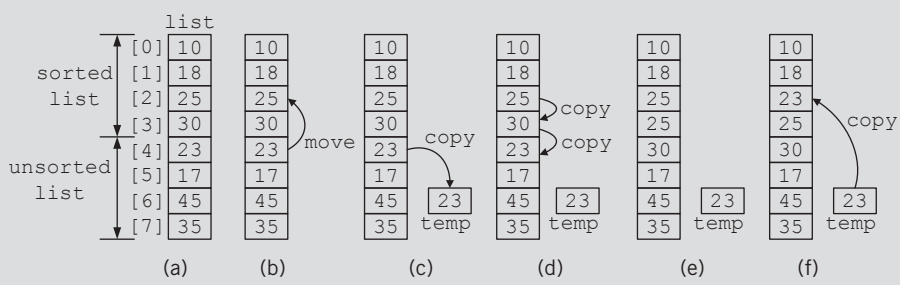


FIGURE 10-5 `list` elements while moving `list[4]` to its proper place

Next, we consider the element `list[4]`, the first element of the unsorted list. Because `list[4] < list[3]`, we need to move the element `list[4]` to its proper location. It follows that element `list[4]` should be moved to `list[2]`, as shown in Figure 10-5(b). To move `list[4]` into `list[2]`, first we copy `list[4]` into `temp`, a temporary memory space—see Figure 10-5(c).

Next, we copy `list[3]` into `list[4]`, and then `list[2]` into `list[3]`, as shown in Figure 10-5(d). After copying `list[3]` into `list[4]` and `list[2]` into `list[3]`, the list is as shown in Figure 10-5(e). Next we copy `temp` into `list[2]`. Figure 10-5(f) shows the resulting list.

Now `list[0]...list[4]` is sorted and `list[5]...list[7]` is unsorted. We repeat this process on the resulting list by moving the first element of the unsorted list into the sorted list in the proper place.

From this discussion, we see that during the sorting phase the array containing the list is divided into two sublists, upper and lower. Elements in the upper sublist are sorted; elements in the lower sublist are to be moved to the upper sublist in their proper places one at a time. We use an index—say, `firstOutOfOrder`—to point to the first element in the lower sublist; that is, `firstOutOfOrder` gives the index of the first element in the unsorted portion of the array. Initially, `firstOutOfOrder` is initialized to `1`.

This discussion translates into the following pseudoalgorithm:

```
for (firstOutOfOrder = 1; firstOutOfOrder < length; firstOutOfOrder++)
  if (list[firstOutOfOrder] is less than list[firstOutOfOrder - 1])
  {
       copy list[firstOutOfOrder] into temp

       initialize location to firstOutOfOrder

       do
       {
           a. move list[location - 1] one array slot down
           b. decrement location by 1 to consider the next element
              sorted of the portion of the array
       }
       while (location > 0 && the element in the upper list at
              location - 1 is greater than temp)
  }
copy temp into list[location]
```

The length of this list is 8; that is, `length = 8`. We initialize `firstOutOfOrder` to `1` (see Figure 10-6).

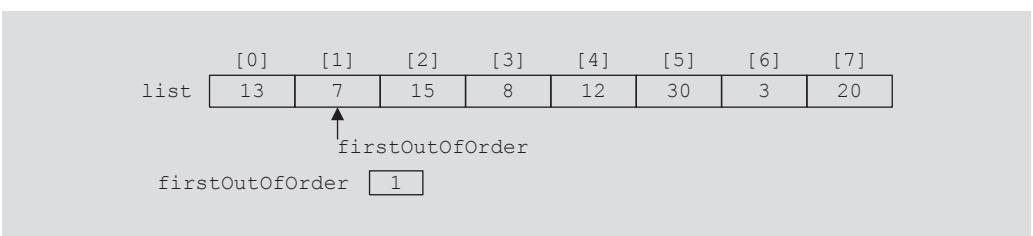


FIGURE 10-6 firstOutOfOrder = 1

Now `list[firstOutOfOrder] = 7`, `list[firstOutOfOrder - 1] = 13` and `7 < 13`, and the expression in the **`if`** statement evaluates to **`true`**, so we execute the body of the **`if`** statement.

```
temp = list[firstOutOfOrder] = 7
location = firstOutOfOrder = 1
```

Next, we execute the **`do...while`** loop.

```
list[1] = list[0] = 13    (copy list[0] into list[1])
location = 0              (decrement location)
```

The **do...while** loop terminates because `location = 0`. We copy `temp` into `list[location]`—that is, into `list[0]`. Figure 10-7 shows the resulting list.

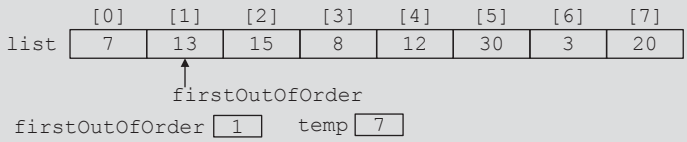


FIGURE 10-7 `list` after the first iteration of insertion sort

Now suppose that we have the list given in Figure 10-8(a).

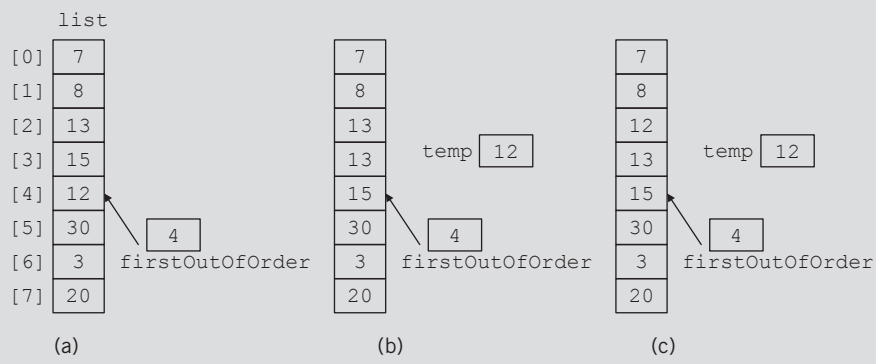


FIGURE 10-8 `list` elements while moving `list[4]` to its proper place

Here `list[0]...list[3]`, or the elements `list[0]`, `list[1]`, `list[2]`, and `list[3]`, are in order. Now `firstOutOfOrder = 4`. Because `list[4] < list[3]`, the element `list[4]`, which is `12`, needs to be moved to its proper location.

As before:

```
temp = list[firstOutOfOrder] = 12
location = firstOutOfOrder = 4
```

First, we copy `list[3]` into `list[4]` and decrement `location` by `1`. Then we copy `list[2]` into `list[3]` and again decrement `location` by `1`. Now the value of `location` is `2`. At this point, the list is as shown in Figure 10-8(b).

Next, because `list[1] < temp`, the **`do...while`** loop terminates. At this point, `location` is 2, so we copy `temp` into `list[2]`. That is, `list[2] = temp = 12`. Figure 10-8(c) shows the resulting list.

Suppose that we have the list given in Figure 10-9.

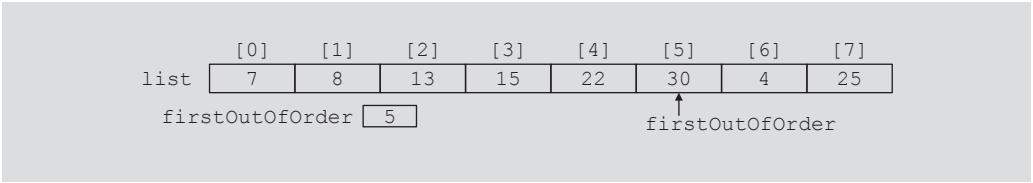


FIGURE 10-9 First out-of-order element is at position 5

Here `list[0]...list[4]`, or the elements `list[0]`, `list[1]`, `list[2]`, `list[3]`, and `list[4]`, are in order. Now `firstOutOfOrder = 5`. Because `list[5] > list[4]`, the **`if`** statement evaluates to **`false`**. So the body of the **`if`** statement does not execute and the next iteration of the **`for`** loop, if any, takes place. Note that this is the case when the `firstOutOfOrder` element is already at the proper place. So we simply need to advance `firstOutOfOrder` to the next array element, if any.

We can repeat this process for the remaining elements of `list` to sort `list`.

The following C++ function implements the previous algorithm:

```
template <class elemType>
void arrayListType<elemType>::insertionSort()
{
    int firstOutOfOrder, location;
    elemType temp;

    for (firstOutOfOrder = 1; firstOutOfOrder < length;
                              firstOutOfOrder++)
        if (list[firstOutOfOrder] < list[firstOutOfOrder - 1])
        {
            temp = list[firstOutOfOrder];
            location = firstOutOfOrder;

            do
            {
                list[location] = list[location - 1];
                location--;
            }
            while (location > 0 && list[location - 1] > temp);

            list[location] = temp;
        }
} //end insertionSort
```

Insertion Sort: Linked List-Based Lists

Insertion sort can also be applied to linked lists. Therefore, this section describes insertion sort for linked lists. Consider the linked list shown in Figure 10-10.

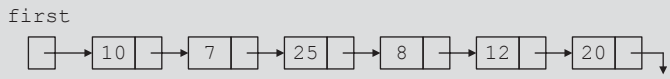


FIGURE 10-10 Linked list

In Figure 10-10, `first` is a pointer to the first node of the linked list.

If the list is stored in an array, we can traverse the list in either direction using an index variable. However, if the list is stored in a linked list, we can traverse the list in only one direction starting at the first node because the links are only in one direction, as shown in Figure 10-10. Therefore, in the case of a linked list, to find the location of the node to be inserted, we do the following. Suppose that `firstOutOfOrder` is a pointer to the node that is to be moved to its proper location, and `lastInOrder` is a pointer to the last node of the sorted portion of the list. For example, see the linked list in Figure 10-11. (We assume that the nodes are in the usual `info-link` form as described in Chapter 5.)

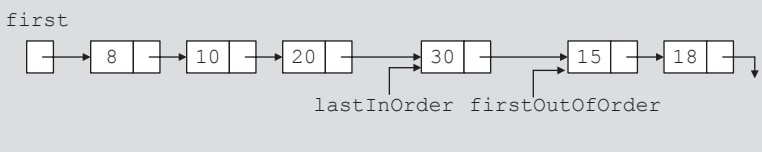


FIGURE 10-11 Linked list and pointers `lastInOrder` and `firstOutOfOrder`

First, we compare the `info` of `firstOutOfOrder` with the `info` of the first node. If the `info` of `firstOutOfOrder` is smaller than the `info` of `first`, then the node `firstOutOfOrder` is to be moved before the first node of the list; otherwise, we search the list starting at the second node to find the location where to move `firstOutOfOrder`. As usual, we search the list using two pointers, for example `current` and `trailCurrent`. The pointer `trailCurrent` points to the node just before `current`. In this case, the node `firstOutOfOrder` is to be moved between `trailCurrent` and `current`. Of course, we also handle any special cases such as an empty list, a list with only one node, or a list in which the node `firstOutOfOrder` is already in the proper place.

This discussion translates into the following algorithm:

```
if (firstOutOfOrder->info is less than first->info)
    move firstOutOfOrder before first
else
{
    set trailCurrent to first
    set current to the second node in the list first->link;

     //search the list
    while (current->info is less than firstOutOfOrder->info)
    {
        advance trailCurrent;
        advance current;
    }

    if (current is not equal to firstOutOfOrder)
    {    //insert firstOutOfOrder between current and trailCurrent
        lastInOrder->link = firstOutOfOrder->link;
        firstOutOfOrder->link = current;
        trailCurrent->link = firstOutOfOrder;
    }
    else   //firstOutOfOrder is already at the first place
        lastInOrder = lastInOrder->link;
}
```

Let us illustrate this algorithm on the list shown in Figure 10-12. We consider several cases.

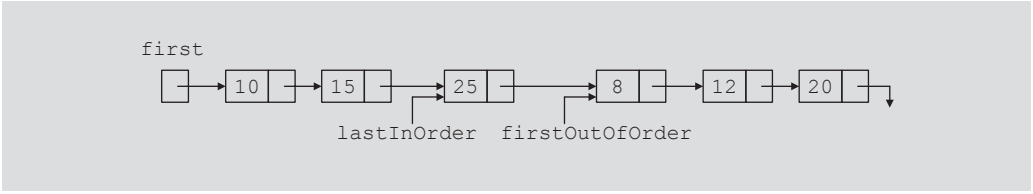


FIGURE 10-12 Linked list and pointers `lastInOrder` and `firstOutOfOrder`

Case 1: Because `firstOutOfOrder->info` is less than `first->info`, the node `firstOutOfOrder` is to be moved before `first`. So we adjust the necessary links, and Figure 10-13 shows the resulting list.

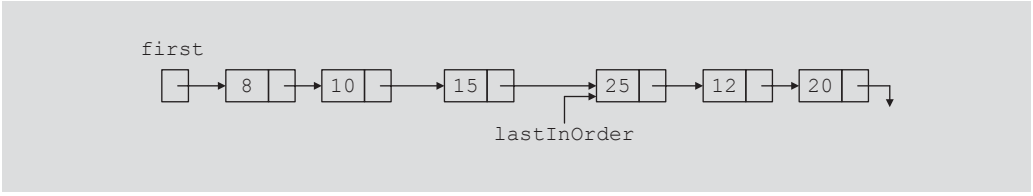


FIGURE 10-13 Linked list after moving the node with `info` 8 to the beginning

Case 2: Consider the list shown in Figure 10-14.

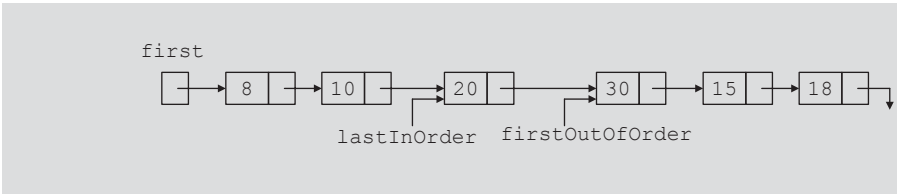


FIGURE 10-14 Linked list and pointers `lastInOrder` and `firstOutOfOrder`

Because `firstOutOfOrder->info` is greater than `first->info`, we search the list to find the place where `firstOutOfOrder` is to be moved. As explained previously, we use the pointers `trailCurrent` and `current` to traverse the list. For this list, these pointers end up at the nodes as shown in Figure 10-15.

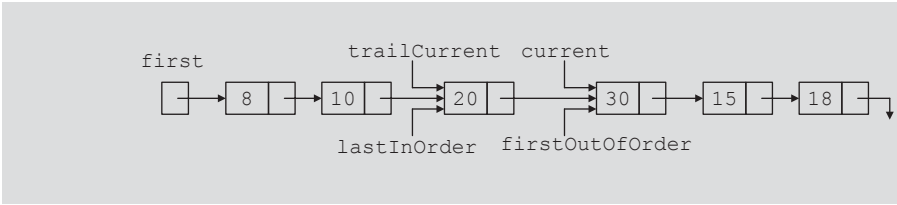


FIGURE 10-15 Linked list and pointers `trailCurrent` and `current`

Because `current` is the same as `firstOutOfOrder`, the node `firstOutOfOrder` is in the right place. So no adjustment of the links is necessary.

Case 3: Consider the list in Figure 10-16.

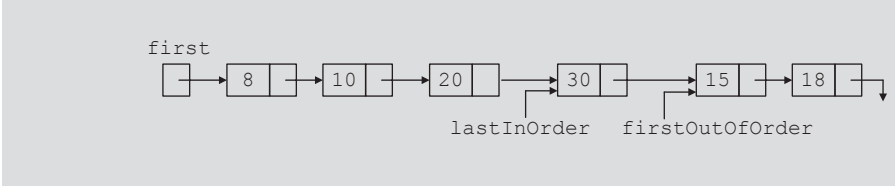


FIGURE 10-16 Linked list and pointers `lastInOrder` and `firstOutOfOrder`

Because `firstOutOfOrder->info` is greater than `first->info`, we search the list to find the place where `firstOutOfOrder` is to be moved. As in Case 2, we use the pointers `trailCurrent` and `current` to traverse the list. For this list, these pointers end up at the nodes as shown in Figure 10-17.

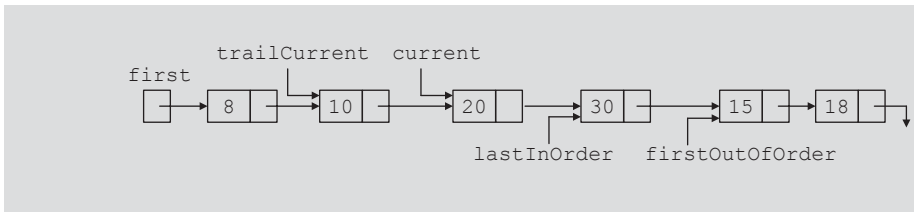


FIGURE 10-17 Linked list and pointers `trailCurrent` and `current`

Now, **`firstOutOfOrder`** is to be moved between **`trailCurrent`** and **`current`**. So we adjust the necessary links and obtain the list as shown in Figure 10-18.

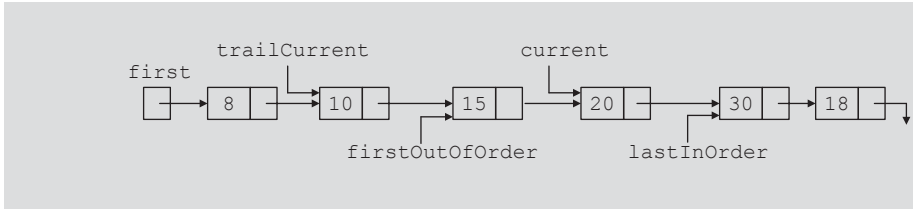


FIGURE 10-18 Linked list after moving `firstOutOfOrder` between `trailCurrent` and `current`

We now write the C++ function, **`linkedInsertionSort`**, to implement the previous algorithm:

```
template <class elemType>
void unorderedLinkedList<elemType>::linkedInsertionSort()
{
    nodeType<elemType> *lastInOrder;
    nodeType<elemType> *firstOutOfOrder;
    nodeType<elemType> *current;
    nodeType<elemType> *trailCurrent;

    lastInOrder = first;

    if (first == NULL)
       cerr << "Cannot sort an empty list." << endl;
    else if (first->link == NULL)
        cout << "The list is of length 1. "
             << "It is already in order." << endl;
    else
        while (lastInOrder->link != NULL)
        {
            firstOutOfOrder = lastInOrder->link;

            if (firstOutOfOrder->info < first->info)
            {
                lastInOrder->link = firstOutOfOrder->link;
                firstOutOfOrder->link = first;
                first = firstOutOfOrder;
            }
```

```
            else
            {
                trailCurrent = first;
                current = first->link;

                while (current->info < firstOutOfOrder->info)
                {
                    trailCurrent = current;
                    current = current->link;
                }

                if (current != firstOutOfOrder)
                {
                    lastInOrder->link = firstOutOfOrder->link;
                    firstOutOfOrder->link = current;
                    trailCurrent->link = firstOutOfOrder;
                }
                else
                    lastInOrder = lastInOrder->link;
            }
        } //end while
} //end linkedInsertionSort
```

We leave it as exercise for you to write a program to test insertion sort. See Programming Exercises 2 and 3 at the end of this chapter.

Analysis: Insertion Sort

Suppose that the list is of length n. If the list is sorted, the number of comparisons is $(n - 1)$ and the number of item assignments is 0. This is the best case. (See Exercise 15 at the end of this chapter.) Now suppose that the list is sorted, but in the reverse order. In this case, it can be checked that the number of comparisons is $(1/2)(n^2 - n)$ and the number of item assignments is $(1/2)(n^2 + 3n) - 2$. This is the worst case. (See Exercise 14 at the end of this chapter.)

Table 10-1 summarizes the average-case behavior of selection and insertion sort. The proofs of the results of insertion sort are given in Appendix F.

TABLE 10-1 Average-case behavior of the selection sort and insertion sort for a list of length n

Algorithm	Number of comparisons	Number of swaps/item assignments
Selection sort	$(1/2)n(n - 1) = O(n^2)$	$3(n - 1) = O(n)$
Insertion sort	$(1/4)n^2 + O(n) = O(n^2)$	$(1/4)n^2 + O(n) = O(n^2)$

Shellsort

In the previous sections, we described selection sort and insertion sort. We noticed that selection sort makes more comparisons and less item movements than insertion sort. Selection sort makes more comparisons because it makes many redundant comparisons. The number of item movements in selection sort is less because each item is moved at most one time. In fact, the number of item movements in insertion sort is considerably more than selection sort because it moves items one position at a time, so to move an item to its final position, it might require many movements.

We can reduce the number of item movements in insertion sort by modifying it. The modified insertion sort that we present next was introduced in 1959 by D. E. Shell and is known as the Shellsort algorithm. This sort is also known as **diminishing-increment sort**.

In Shellsort, the elements of the list are viewed as sublists at a particular distance. Each sublist is sorted, so that elements that are far apart move closer to their final position. For example, suppose that you have a list of 15 elements, as shown in Figure 10-19. First we view the list as 7 sublists, that is, we sort the elements at a distance of 7. Note that several elements have moved closer to their final position. For example, `2`, `19`, and `60` are closer to their final position. In the next iteration, we sort the elements at a distance of 4, as shown in Figure 10-19(b). Finally, we sort the elements at a distance of 1, that is, the entire list is sorted. Figure 10-19(c) shows the elements before and after the final sorting phase.

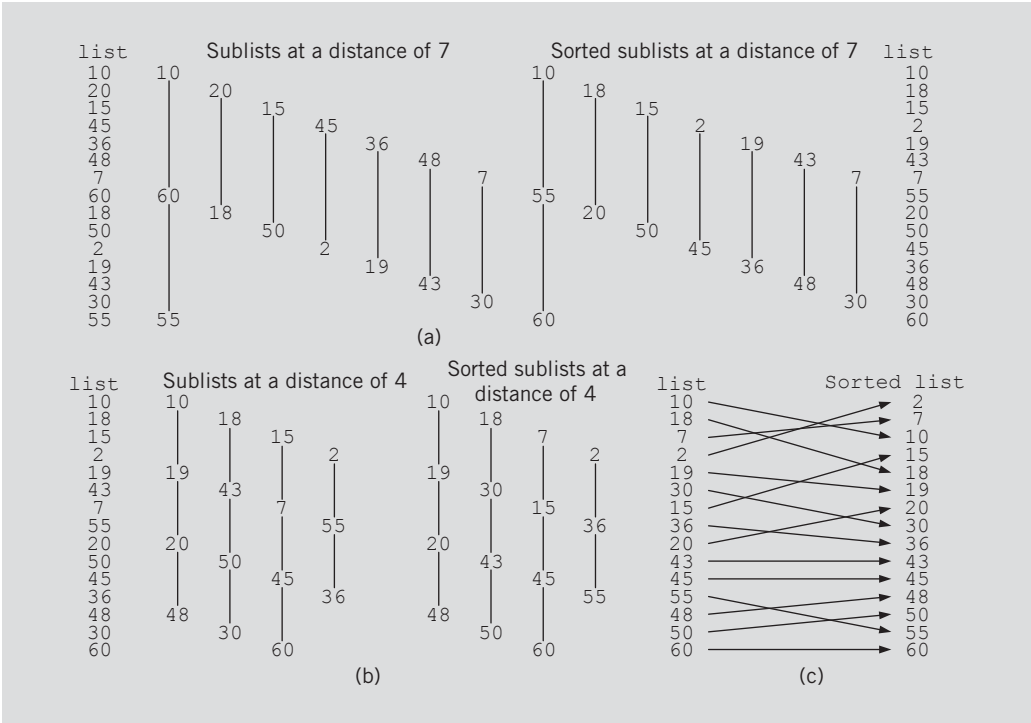


FIGURE 10-19 Lists during Shellsort

In Figure 10-19, we sorted the elements at a distance of 7, 4, and then 1. The sequence 1, 4, 7 is called the **increment sequence**. How do we choose an increment sequence? In general, this question cannot be answered satisfactorily. The literature provides a discussion of various increment sequences, and some have been found to be useful. Typically, the increment sequences are chosen to decrease roughly geometrically so that the number of increments is logarithmic in the size of the list. For example, if the number of increments is about one-half of the previous increment, then we need at most 20 increments to sort a list of 1 million elements. However, using as few increments as possible is desirable.

D. E. Knuth recommended the increment sequence 1, 4, 13, 40, 121, 364, 1093, 3280, ... The ratio between successive increments is about one-third. In fact, the ith increment = 3 • (i – 1)th increment + 1. There are many other increment sequences that could lead to more efficient sorts. However, for large lists, it is difficult to get a better performance by more than 20% than the increment sequence recommended by Knuth.

There are certain increment sequences that must be avoided. For example, the increment sequence, 1, 2, 4, 8, 16, 32, 64, 128, 256, . . . , is likely to lead to a bad performance because elements at odd positions are not compared with the elements at even positions until the final pass. We will use the increment sequence suggested by Knuth in the Shellsort algorithm we implement.

This following function implements the Shellsort algorithm:

```
template <class elemType>
void arrayListType<elemType>::shellSort()
{
    int inc;

    for (inc = 1; inc < (length - 1) / 9; inc = 3 * inc + 1);

    do
    {
        for (int begin = 0; begin < inc; begin++)
            intervalInsertionSort(begin, inc);

        inc = inc / 3;
    }
    while (inc > 0);
} //end shellSort
```

In the function `shellSort`, we use the function `intervalInsertionSort`, which is a modified version of insertion sort for array-based lists, discussed earlier in this chapter. In the `intervalInsertionSort`, the sublist starts at the variable `begin`, and the increment between successive elements is given by the variable `inc` instead of 1. We leave the details of the function `intervalInsertionSort` as an exercise for you.

The analysis of the Shellsort is difficult to obtain. In fact, to date, good estimates of the number of comparisons and item movements have been obtained only under special conditions depending on the increment sequence. Empirical studies suggest that for large lists of size n, the number of moves is in the range of $n^{1.25}$ to $1.6n^{1.25}$, which is a considerable improvement over insertion sort.

Lower Bound on Comparison-Based Sort Algorithms

The previous sections discussed selection sort and insertion sort, and noted that the average-case behavior of these algorithms is $O(n^2)$. Both of these algorithms are comparison-based algorithms; that is, the lists are sorted by comparing their respective keys. Before discussing any additional sorting algorithms, let us discuss the best-case scenario for the comparison-based sorting algorithms.

We can trace the execution of a comparison-based algorithm using a graph called a **comparison tree**. Let L be a list of n distinct elements, where $n > 0$. For any j and k, where $1 \leq j, k \leq n$, either $L[j] < L[k]$ or $L[j] > L[k]$. Because each comparison of the keys has two outcomes, the comparison tree is a binary tree. While drawing this figure, we draw each comparison as a circle, called a **node**. The node is labeled as j:k, representing the comparison of $L[j]$ with $L[k]$. If $L[j] < L[k]$, follow the left branch; otherwise, follow the right branch. Figure 10-20 shows the comparison tree for a list of length 3. (In Figure 10-20, the rectangle, called **leaf**, represents the final ordering of the nodes.)

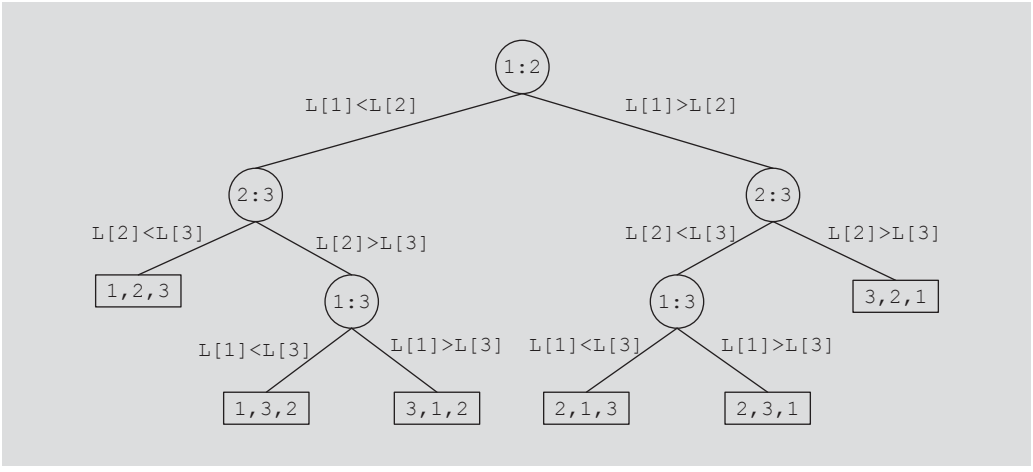


FIGURE 10-20 Comparison tree for sorting three items

We call the top node in the figure the **root** node. The straight line that connects the two nodes is called a **branch**. A sequence of branches from a node, x, to another node, y, is called a **path** from x to y.

Associated with each path from the root to a leaf is a unique permutation of the elements of L. This uniqueness follows because the sort algorithm only moves the data and makes comparisons. Furthermore, the data movement on any path from the root to a leaf is the same regardless of the initial inputs. For a list of n elements, $n > 0$, there are $n!$ different permutations. Any one of these $n!$ permutations might be the correct ordering of L. Thus, the comparison tree must have at least $n!$ leaves.

Let us consider the worst case for all comparison-based sorting algorithms. We state the following result without proof.

Theorem: Let L be a list of n distinct elements. Any sorting algorithm that sorts L by comparison of the keys only, in its worst case, makes at least $O(n\log_2 n)$ key comparisons.

As analyzed in the previous sections, both selection sort and insertion sort are of the order $O(n^2)$. The remainder of this chapter discusses sorting algorithms that, on average, are of the order $O(n\log_2 n)$.

Quicksort: Array-Based Lists

In the previous section, we noted that the lower bound on comparison-based algorithms is $O(n\log_2 n)$. The sorting algorithms selection sort and insertion sort, discussed earlier in this chapter, are $O(n^2)$. In this and the next two sections, we discuss sorting algorithms that are usually of the order $O(n\log_2 n)$. The first algorithm is quicksort.

Quicksort uses the divide-and-conquer technique to sort a list. The list is partitioned into two sublists, and the two sublists are then sorted and combined into one list in such a way so that the combined list is sorted. Thus, the general algorithm is as follows:

```
if (the list size is greater than 1)
{
   a. Partition the list into two sublists, say lowerSublist and upperSublist.
   b. Quicksort lowerSublist.
   c. Quicksort upperSublist.
   d. Combine the sorted lowerSublist and sorted upperSublist.
}
```

After partitioning the list into two sublists—`lowerSublist` and `upperSublist`—these two sublists are sorted using quicksort. In other words, we use *recursion* to implement quicksort.

Quicksort described here is for array-based lists. The algorithm for linked lists can be developed in a similar manner and is left as an exercise for you. See Programming Exercise 7 at the end of this chapter.

In quicksort, the list is partitioned in such a way that combining the sorted `lowerSublist` and `upperSublist` is trivial. Therefore, in quicksort, all the sorting work is done in partitioning the list. Because all the sorting work occurs during the partition, we first describe the partition procedure in detail.

To partition the list into two sublists, first we choose an element of the list called the **`pivot`**. The `pivot` is used to divide the list into two sublists: the `lowerSublist` and the `upperSublist`. The elements in the `lowerSublist` are smaller than the `pivot`, and the elements in the `upperSublist` are greater than the `pivot`. For example, consider the list in Figure 10-21.

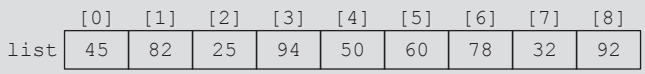


FIGURE 10-21 `list` before the partition

There are several ways to determine the `pivot`. However, the `pivot` is chosen so that, it is hoped, the `lowerSublist` and `upperSublist` are of nearly equal size. For illustration purposes, let us choose the middle element of the list as the `pivot`. The partition procedure that we describe partitions this list using the `pivot` as the middle element, in our case `50`, as shown in Figure 10-22.

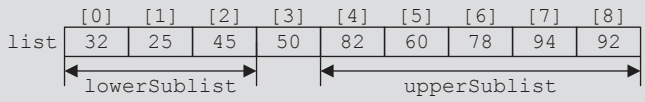


FIGURE 10-22 `list` after the partition

From Figure 10-22, it follows that after partitioning `list` into `lowerSublist` and `upperSublist`, the `pivot` is in the right place. Thus, after sorting `lowerSublist` and `upperSublist`, combining the two sorted sublists is trivial.

The partition algorithm is as follows: (We assume that `pivot` is chosen as the middle element of the list.)

1. Determine the `pivot`, and swap the `pivot` with the first element of the list.

   Suppose that the index `smallIndex` points to the last element smaller than the `pivot`. The index `smallIndex` is initialized to the first element of the list.
2. For the remaining elements in the list (starting at the second element)
   If the current element is smaller than the `pivot`
   a. Increment `smallIndex`.
   b. Swap the current element with the array element pointed to by `smallIndex`.
3. Swap the first element, that is, the `pivot`, with the array element pointed to by `smallIndex`.

Step 2 can be implemented using a `for` loop, with the loop starting at the second element of the list.

Step 1 determines the `pivot` and moves the `pivot` in the first array position. During the execution of Step 2, the elements of the list get arranged as shown in Figure 10-23. (Suppose the name of the array containing the list elements is `list`.)

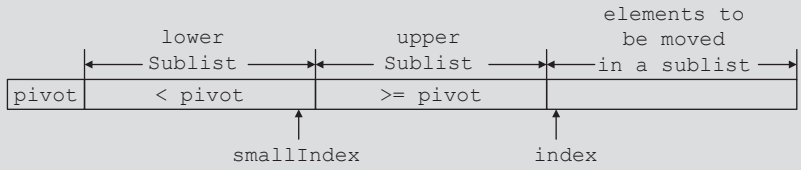


FIGURE 10-23 List during the execution of Step 2

As shown in Figure 10-23, the `pivot` is in the first array position, elements in `lowerSublist` are less than the `pivot`, and elements in the `upperSublist` are greater than or equal to the `pivot`. The variable `smallIndex` contains the index of the last element of `lowerSublist` and the variable `index` contains the index of the next element that needs to be moved either in `lowerSublist` or in `upperSublist`. As explained in Step 2, if the next element of the list (that is, `list[index]`) is less than the `pivot`, we advance `smallIndex` to the next array position and swap `list[index]` with `list[smallIndex]`. Next we illustrate Step 2.

Suppose that `list` is as given in Figure 10-24.

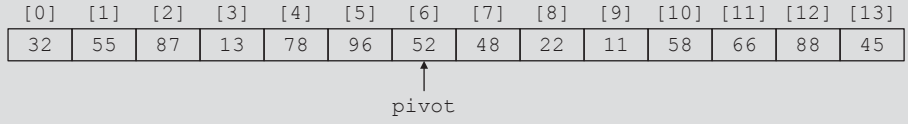


FIGURE 10-24 `list` before sorting

For the list in Figure 10-24, the `pivot` is at position `6`. After moving the `pivot` at the first array position, the list is shown in Figure 10-25. (Notice that in Figure 10-25, `52` is swapped with `32`.)

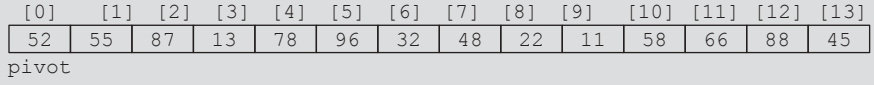


FIGURE 10-25 List after moving `pivot` at the first array position

Suppose that after executing Step 2 a few times, the list is as shown in Figure 10-26.

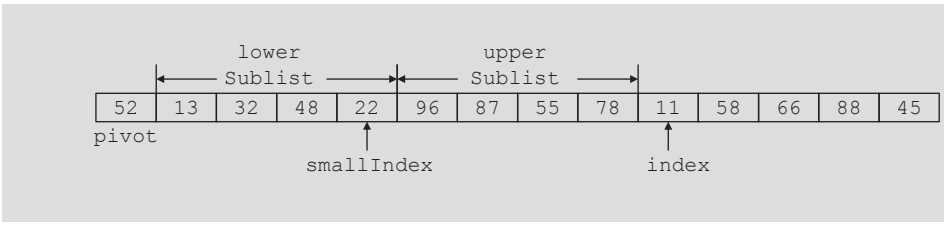


FIGURE 10-26 List after a few iterations of Step 2

As shown in Figure 10-26, the next element of the list that needs to be moved in a sublist is indicated by **index**. Because **list[index] < pivot**, we need to move the element **list[index]** in **lowerSublist**. To do so, we first advance **smallIndex** to the next array position and then swap **list[smallIndex]** with **list[index]**. The resulting list is shown in Figure 10-27. (Notice that **11** is swapped with **96**.)

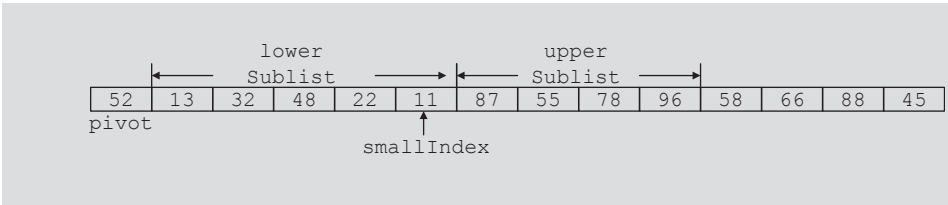


FIGURE 10-27 List after moving 11 into lowerSublist

Now consider the list in Figure 10-28.

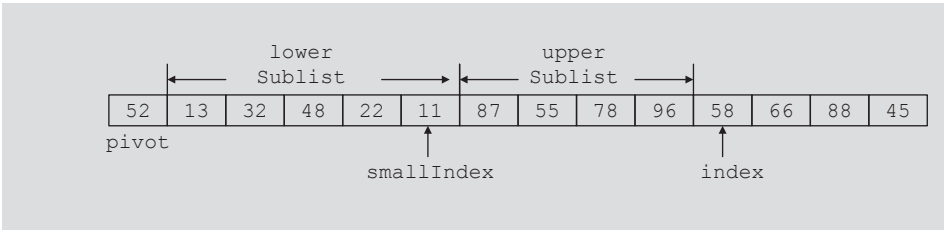


FIGURE 10-28 List before moving 58 into a sublist

For the list in Figure 10-28, **list[index]** is **58**, which is greater than the **pivot**. Therefore, **list[index]** is to be moved in **upperSublist**. This is accomplished by leaving **58** at its position and increasing the size of **upperSublist**, by one, to the next array position. After moving **58** into **upperSublist**, the list is as shown in Figure 10-29.

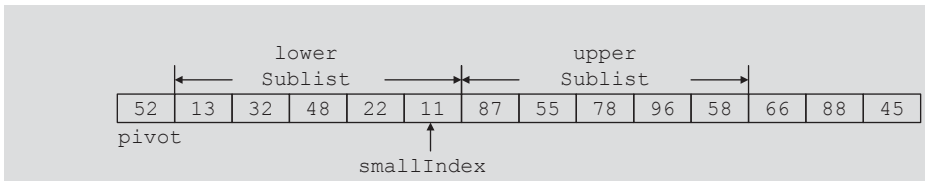


FIGURE 10-29 List after moving 58 into upperSublist

After moving the elements that are less than the **pivot** into **lowerSublist** and elements that are greater than the **pivot** into **upperSublist** (that is, after completely executing Step 2), the resulting list is as shown in Figure 10-30.

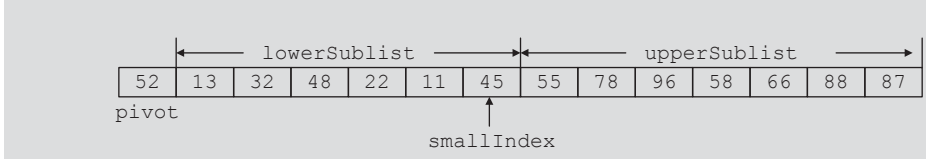


FIGURE 10-30 List elements after arranging into lowerSublist and upperSublist

Next, we execute Step 3 and move **52**, the **pivot**, to the proper position in the list. This is accomplished by swapping **52** with **45**. The resulting list is shown in Figure 10-31.

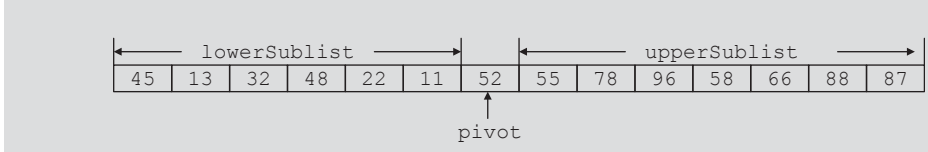


FIGURE 10-31 List after swapping 52 with 45

As shown in Figure 10-31, the preceding algorithm, Steps 1, 2, and 3, partitions the list into two sublists such that the elements less than the **pivot** are in **lowerSublist** and elements greater than or equal to the **pivot** are in **upperSublist**.

To partition the list into the lower and upper sublists, we only need to keep track of the last element of the **lowerSublist** and the next element of the list that needs to be moved either in **lowerSublist** or in **upperSublist**. In fact, **upperSublist** is between the two indices **smallIndex** and **index**.

We now write the function, **partition**, to implement the preceding partition algorithm. After rearranging the elements of the list, the function returns the location of the

pivot so that we can determine the starting and ending locations of the sublists. Also, because the function partition will be a member of the class, it has direct access to the array containing the list. Thus, to partition a list, we need to pass only the starting and ending indices of the list.

```
template <class elemType>
int arrayListType<elemType>::partition(int first, int last)
{
    elemType pivot;

    int index, smallIndex;

    swap(first, (first + last) / 2);

    pivot = list[first];
    smallIndex = first;

    for (index = first + 1; index <= last; index++)
        if (list[index] < pivot)
        {
            smallIndex++;
            swap(smallIndex, index);
        }

    swap(first, smallIndex);

    return smallIndex;
}
```

As you can see from the definition of the function partition, certain elements of the list need to be swapped. The following function, swap, accomplishes this task:

```
template <class elemType>
void arrayListType<elemType>::swap(int first, int second)
{
    elemType temp;

    temp = list[first];
    list[first] = list[second];
    list[second] = temp;
}
```

Once the list is partitioned into lowerSublist and upperSublist, we again apply the quicksort method to sort the two sublists. Because both sublists are sorted using the same quicksort algorithm, the easiest way to implement this algorithm is to use recursion. Therefore, this section gives the recursive version of quicksort. As explained previously, after rearranging the elements of the list, the function partition returns the index of the pivot so that the starting and ending indices of the sublists can be determined.

Given the starting and ending indices of a list, the following function, recQuickSort, implements the recursive version of quicksort:

```
template <class elemType>
void arrayListType<elemType>::recQuickSort(int first, int last)
{
    int pivotLocation;

    if (first < last)
    {
        pivotLocation = partition(first, last);
        recQuickSort(first, pivotLocation - 1);
        recQuickSort(pivotLocation + 1, last);
    }
}
```

Finally, we write the quicksort function, `quickSort`, that calls the function `recQuickSort` of the original list:

```
template <class elemType>
void arrayListType<elemType>::quickSort()
{
    recQuickSort(0, length -1);
}
```

We leave it as an exercise for you to write a program to test quicksort. See Programming Exercise 7 at the end of this chapter.

Analysis: Quicksort

Table 10-2 summarizes the behavior of quicksort for a list of length n. (The proofs of these results are given in Appendix F.)

TABLE 10-2 Analysis of quicksort for a list of length n

	Number of comparisons	Number of swaps
Average case	$1.39n\log_2 n + O(n) = O(n\log_2 n)$	$0.69n\log_2 n + O(n) = O(n\log_2 n)$
Worst case	$(1/2)(n^2 - n) = O(n^2)$	$(1/2)n^2 + (3/2)n - 2 = O(n^2)$

Mergesort: Linked List-Based Lists

In the previous section, we described quicksort and stated that the average-case behavior of quicksort is $O(n\log_2 n)$. However, the worst-case behavior of quicksort is $O(n^2)$. This section describes a sorting algorithm whose behavior is always $O(n\log_2 n)$.

Like quicksort, mergesort uses the divide-and-conquer technique to sort a list. Mergesort also partitions the list into two sublists, sorts the sublists, and then combines the sorted sublists into one sorted list. This section describes mergesort for linked list-based lists.

We leave it for you to develop mergesort for array-based lists, which can be done by using the techniques described for linked lists.

Mergesort and quicksort differ in how they partition the list. As discussed earlier, quicksort first selects an element in the list, called `pivot`, and then partitions the list so that the elements in one sublist are less than `pivot` and the elements in the other sublist are greater than or equal to `pivot`. By contrast, mergesort divides the list into two sublists of nearly equal size. For example, consider the list whose elements are as follows:

```
list: 35  28  18  45  62  48  30  38
```

Mergesort partitions this list into two sublists as follows:

```
first sublist: 35  28  18  45
second sublist: 62  48  30  38
```

The two sublists are sorted using the same algorithm (that is, mergesort) used on the original list. Suppose that we have sorted the two sublists. That is, suppose that the lists are now as follows:

```
first sublist: 18  28  35  45
second sublist: 30  38  48  62
```

Next, mergesort combines, that is, merges, the two sorted sublists into one sorted list.

Figure 10-32 further illustrates the mergesort process.

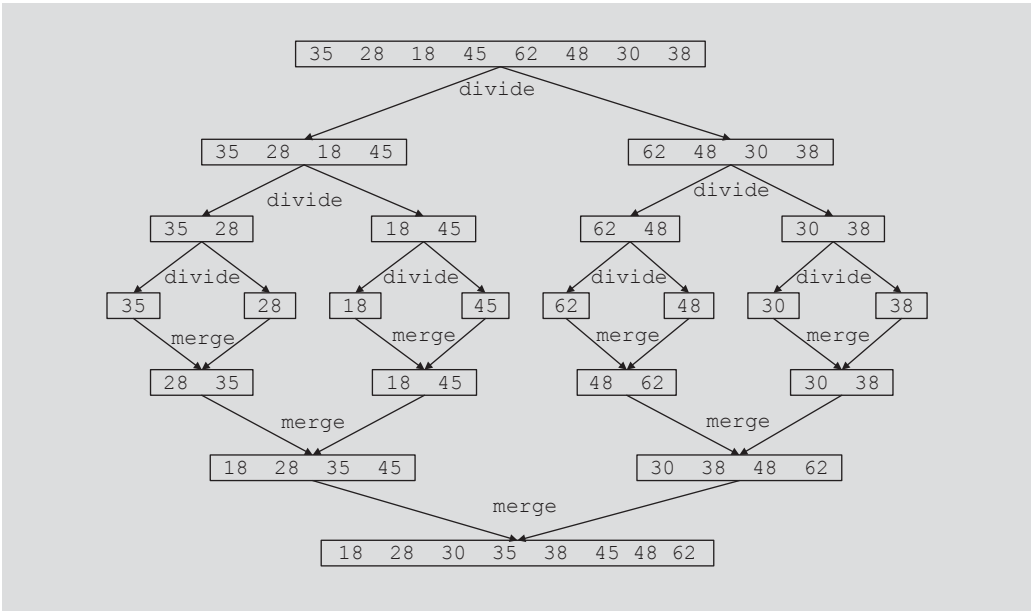


FIGURE 10-32 Mergesort algorithm

From Figure 10-32, it is clear that in mergesort, most of the sorting work is done in merging the sorted sublists.

The general algorithm for mergesort is as follows:

```
if the list is of a size greater than 1
{
   1. Divide the list into two sublists.
   2. Mergesort the first sublist.
   3. Mergesort the second sublist.
   4. Merge the first sublist and the second sublist.
}
```

As remarked previously, after dividing the list into two sublists—the first sublist and the second sublist—these two sublists are sorted using mergesort. In other words, we use recursion to implement mergesort.

We next describe the necessary algorithm to:

- Divide the list into two sublists of nearly equal size.
- Mergesort both sublists.
- Merge the sorted sublists.

Divide

Because data is stored in a linked list, we do not know the length of the list. Furthermore, a linked list is not a random access data structure. Therefore, to divide the list into two sublists, we need to find the middle node of the list.

Consider the list in Figure 10-33.

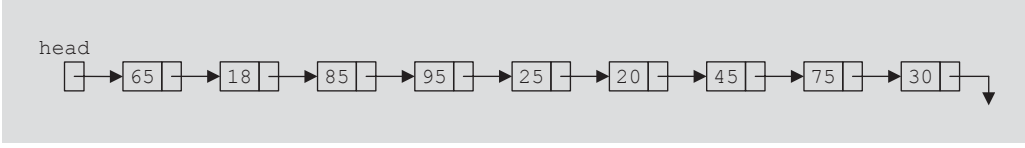


FIGURE 10-33 Unsorted linked list

To find the middle of the list, we traverse the list with two pointers—say, `middle` and `current`. The pointer `middle` is initialized to the first node of the list. Because this list has more than two nodes, we initialize `current` to the third node. (Recall that we sort the list only if it has more than one element because a list of size 1 is already sorted. Also, if the list has only two nodes, we set `current` to `NULL`.) Consider the list shown in Figure 10-34.

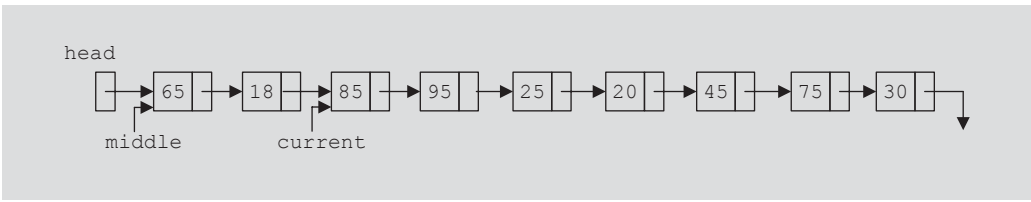


FIGURE 10-34 `middle` and `current` before traversing the list

Every time we advance **`middle`** by one node, we advance **`current`** by one node. After advancing **`current`** by one node, if **`current`** is not **`NULL`**, we again advance **`current`** by one node. That is, for the most part, every time **`middle`** advances by one node, **`current`** advances by two nodes. Eventually, **`current`** becomes **`NULL`** and **`middle`** points to the last node of the first sublist. For example, for the list in Figure 10-34, when **`current`** becomes **`NULL`**, **`middle`** points to the node with **`info`** `25` (see Figure 10-35).

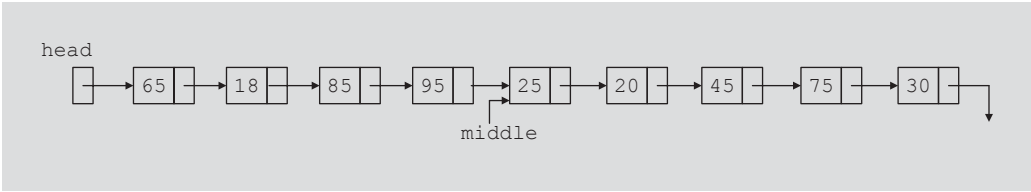


FIGURE 10-35 `middle` after traversing the list

It is now easy to divide the list into two sublists. First, using the link of **`middle`**, we assign a pointer to the node following **`middle`**. Then we set the link of **`middle`** to **`NULL`**. Figure 10-36 shows the resulting sublists.

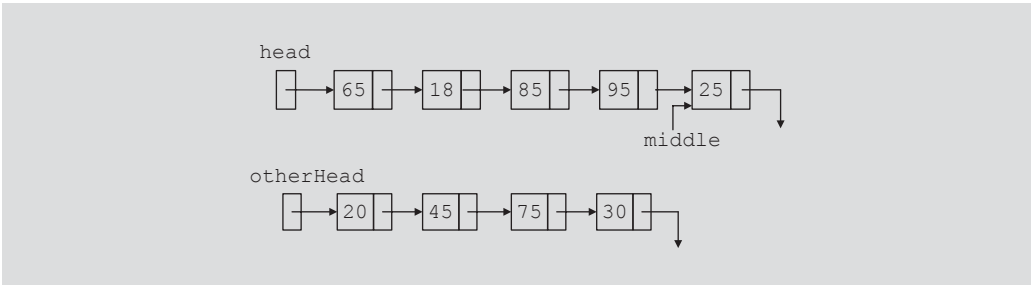


FIGURE 10-36 List after dividing it into two lists

This discussion translates into the following C++ function, **`divideList`**:

```
template <class Type>
void unorderedLinkedList<Type>::
                divideList(nodeType<Type>* first1,
                           nodeType<Type>* &first2)
```

```
{
    nodeType<Type>* middle;
    nodeType<Type>* current;

    if (first1 == NULL)   //list is empty
        first2 = NULL;
    else if (first1->link == NULL)  //list has only one node
        first2 = NULL;
    else
    {
        middle = first1;
        current = first1->link;

        if (current != NULL)    //list has more than two nodes
            current = current->link;
        while (current != NULL)
        {
            middle = middle->link;
            current = current->link;
            if (current != NULL)
                current = current->link;
        } //end while

        first2 = middle->link; //first2 points to the first
                               //node of the second sublist
        middle->link = NULL;   //set the link of the last node
                               //of the first sublist to NULL
    } //end else
} //end divideList
```

Now that we know how to divide a list into two sublists of nearly equal size, next we focus on merging the sorted sublists. Recall that, in mergesort, most of the sorting work is done in merging the sorted sublists.

Merge

Once the sublists are sorted, the next step in mergesort is to merge the sorted sublists. Sorted sublists are merged into a sorted list by comparing the elements of the sublists, and then adjusting the references of the nodes with the smaller `info`. Let us illustrate this procedure on the sublists shown in Figure 10-37. Suppose that `first1` points to the first node of the first sublist, and `first2` points to the first node of the second sublist.

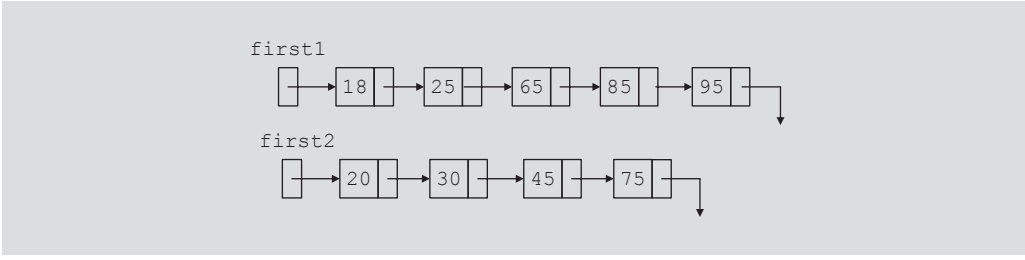


FIGURE 10-37 Sublists before merging

We first compare the **info** of the first node of each of the two sublists to determine the first node of the merged list. We set **newHead** to point to the first node of the merged list. We also use the pointer **lastMerged** to keep track of the last node of the merged list. The pointer of the first node of the sublist with the smaller node then advances to the next node of that sublist. Figure 10-38 shows the sublist of Figure 10-37 after setting **newHead** and **lastMerged** and advancing **first1**.

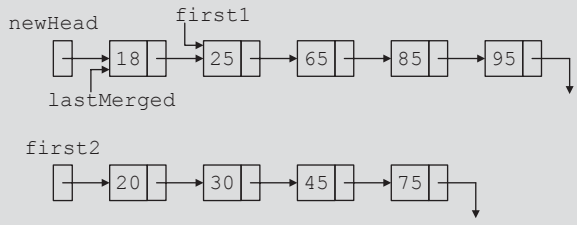


FIGURE 10-38 Sublists after setting `newHead` and `lastMerged` and advancing `first1`

In Figure 10-38, **first1** points to the first node of the first sublist that is yet to be merged with the second sublist. So we again compare the nodes pointed to by **first1** and **first2**, and adjust the link of the smaller node and the last node of the merged list so as to move the smaller node to the end of the merged list. For the sublists shown in Figure 10-38, after adjusting the necessary links, we have Figure 10-39.

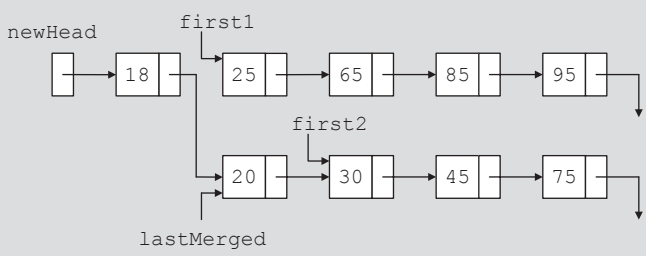


FIGURE 10-39 Merged list after putting the node with `info 20` at the end of the merged list

We continue this process for the remaining elements of both sublists. Every time we move a node to the merged list, we advance either **first1** or **first2** to the next node. Eventually, either **first1** or **first2** becomes **NULL**. If **first1** becomes **NULL**, the first sublist is exhausted first, so we attach the remaining nodes of the second sublist at the end of the partially merged list. If **first2** becomes **NULL**, the second sublist is exhausted first, so we attach the remaining nodes of the first sublist at the end of the partially merged list.

Following this discussion, we can now write the C++ function, `mergeList`, to merge the two sorted sublists. The references (that is, addresses) of the first nodes of the sublists are passed as parameters to the function `mergeList`.

```
template <class Type>
nodeType<Type>* unorderedLinkedList<Type>::
                  mergeList(nodeType<Type>* first1,
                            nodeType<Type>* first2)
{
    nodeType<Type> *lastSmall; //pointer to the last node of
                               //the merged list
    nodeType<Type> *newHead;   //pointer to the merged list

    if (first1 == NULL)   //the first sublist is empty
        return first2;
    else if (first2 == NULL)   //the second sublist is empty
        return first1;
    else
    {
        if (first1->info < first2->info) //compare the first nodes
        {
            newHead = first1;
            first1 = first1->link;
            lastSmall = newHead;
        }
        else
        {
            newHead = first2;
            first2 = first2->link;
            lastSmall = newHead;
        }

        while (first1 != NULL && first2 != NULL)
        {
            if (first1->info < first2->info)
            {
                lastSmall->link = first1;
                lastSmall = lastSmall->link;
                first1 = first1->link;
            }
            else
            {
                lastSmall->link = first2;
                lastSmall = lastSmall->link;
                first2 = first2->link;
            }
        } //end while

        if (first1 == NULL) //first sublist is exhausted first
            lastSmall->link = first2;
```

```
        else                //second sublist is exhausted first
            lastSmall->link = first1;

        return newHead;
    }
}//end mergeList
```

Finally, we write the recursive mergesort function, `recMergeSort`, which uses the `divideList` and `mergeList` functions to sort a list. The reference of the first node of the list to be sorted is passed as a parameter to the function `recMergeSort`.

```
template <class Type>
void unorderedLinkedList<Type>::recMergeSort(nodeType<Type>* &head)
{
    nodeType<Type> *otherHead;

    if (head != NULL)  //if the list is not empty
        if (head->link != NULL) //if the list has more than one node
        {
            divideList(head, otherHead);
            recMergeSort(head);
            recMergeSort(otherHead);
            head = mergeList(head, otherHead);
        }
} //end recMergeSort
```

We can now give the definition of the function `mergeSort`, which should be included as a `public` member of the `class unorderedLinkedList`. (Note that the functions `divideList`, `merge`, and `recMergeSort` can be included as `private` members of the `class unorderedLinkedList` because these functions are used only to implement the function `mergeSort`.) The function `mergeSort` calls the function `recMergeSort` and passes `first` to this function. It also sets `last` to point to the last node of the list. The definition of the function `mergeSort` is as follows:

```
template<class Type>
void unorderedLinkedList<Type>::mergeSort()
{
    recMergeSort(first);

    if (first == NULL)
        last = NULL;
    else
    {
        last = first;
        while (last->link != NULL)
            last = last->link;
    }
} //end mergeSort
```

We leave it as an exercise for you to write a program to test mergesort. See Programming Exercise 10 at the end of this chapter.

Analysis: Mergesort

Suppose that L is a list of n elements, where $n > 0$. Suppose that n is a power of 2, that is, $n = 2^m$ for some nonnegative integer m, so that we can divide the list into two sublists, each of size $n / 2 = 2^m / 2 = 2^{m-1}$. Moreover, each sublist can also be divided into two sublists of the same size. Each call to the function `recMergeSort` makes two recursive calls to the function `recMergeSort` and each call divides the sublist into two sublists of the same size. Suppose that $m = 3$, that is, $n = 2^3 = 8$. So the length of the original list is 8. The first call to the function `recMergeSort` divides the original list into two sublists, each of size 4. The first call then makes two recursive calls to the function `recMergeSort`. Each of these recursive calls divides each sublist, of size 4, into two sublists, each of size 2. We now have 4 sublists, each of size 2. The next set of recursive calls divides each sublist, of size 2, into sublists of size 1. So we now have 8 sublists, each of size 1. It follows that the exponent 3 in 2^3 indicates the level of the recursion, as shown in Figure 10-40.

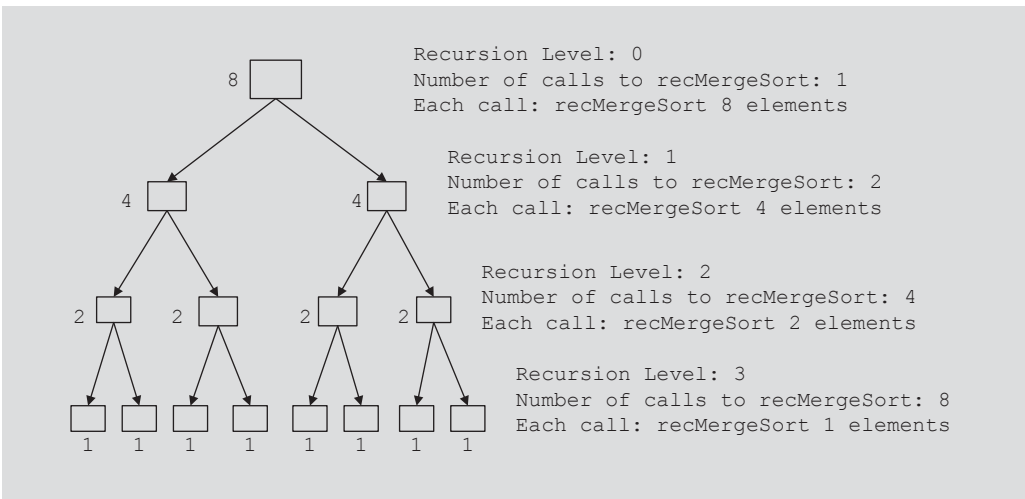


FIGURE 10-40 Levels of recursion levels to `recMergeSort` for a list of length 8

Let us consider the general case when $n = 2^m$. Note that the number of recursion levels is m. Also, note that to merge a sorted list of size s with a sorted list of size t, the maximum number of comparisons is $s + t - 1$.

Consider the function `mergeList`, which merges two sorted lists into a sorted list. Note that this is where the actual work, comparisons and assignments, is done. The initial call to the function `recMergeSort`, at level 0, produces two sublists, each of size $n / 2$. To merge these two lists, after they are sorted, the maximum number of comparisons is $n / 2 + n / 2 - 1 = n - 1 = O(n)$. At level 1, we merge two sets of sorted lists, where each sublist is of size $n / 4$. To merge two sorted sublists, each of size $n / 4$, we need at most $n / 4 + n / 4 - 1 = n / 2 - 1$ comparisons. Thus, at level 1 of the recursion, the number of comparisons is $2(n / 2 - 1) = n - 2 = O(n)$. In general, at level k of the recursion, there are a total of 2^k calls

to the function `mergeList`. Each of these calls merge two sublists, each of size $n / 2^{k+1}$, which requires a maximum of $n / 2^k - 1$ comparisons. Thus, at level k of the recursion, the maximum number of comparisons is $2^k (n / 2^k - 1) = n - 2^k = O(n)$. It now follows that the maximum number of comparisons at each level of the recursion is $O(n)$. Because the number of levels of the recursion is m, the maximum number of comparisons made by mergesort is $O(nm)$. Now $n = 2^m$ implies that $m = \log_2 n$. Hence, the maximum number of comparisons made by mergesort is $O(n \log_2 n)$.

If $W(n)$ denotes the number of key comparisons in the worst case to sort L, then $W(n) = O(n \log_2 n)$.

Let $A(n)$ denote the number of key comparisons in the average case. In the average case, during the merge process one of the sublists will exhaust before the other list. From this, it follows that on average merging of two sorted sublists of combined size n, the number of comparisons will be less than $n - 1$. On average, it can be shown that the number of comparisons for mergesort is given by the following equation: If n is a power of 2, $A(n) = n \log_2 n - 1.25n = O(n \log_2 n)$. This is also a good approximation when n is not a power of 2.

NOTE

We can also obtain an analysis of mergesort by constructing and solving certain equations as follows. As noted before, in mergesort, all the comparisons are made in the method `mergeList`, which merges two sorted sublists. If one sublist is of size s and the other sublist is of size t, merging these lists would require at most $s + t - 1$ comparisons in the worst case. Hence,

$$W(n) = W(s) + W(t) + s + t - 1$$

Note that $s = n / 2$ and $t = n / 2$. Suppose that $n = 2^m$. Then $s = 2^{m-1}$ and $t = 2^{m-1}$. It follows that $s + t = n$. Hence,

$$W(n) = W(n / 2) + W(n / 2) + n - 1 = 2\ W(n / 2) + n - 1,\ n > 0$$

Also,

$$W(1) = 0$$

It is known that when n is a power of 2, $W(n)$ is given by the following equation:

$$W(n) = n \log_2 n - (n - 1) = O(n \log_2 n)$$

10

Heapsort: Array-Based Lists

In an earlier section, we described the quicksort algorithm for contiguous lists, that is, array-based lists. We remarked that, on average, quicksort is of the order $O(n\log_2 n)$. However, in the worst case, quicksort is of the order $O(n^2)$. This section describes another algorithm, the **heapsort**, for array-based lists. This algorithm is of order $O(n \log_2 n)$ even in the worst case, therefore overcoming the worst case of the quicksort.

Definition: A **heap** is a list in which each element contains a key, such that the key in the element at position k in the list is at least as large as the key in the element at position $2k + 1$ (if it exists) and $2k + 2$ (if it exists).

Recall that, in C++ the array index starts at `0`. Therefore, the element at position k is in fact the $k + 1th$ element of the list.

Consider the list in Figure 10-41.

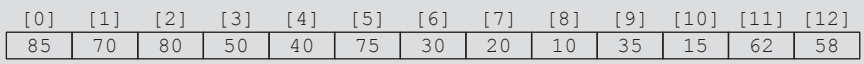


FIGURE 10-41 A heap

It can be checked that the list in Figure 10-41 is in a heap. For example, consider `list[3]`, which is `50`. The elements at position `list[7]` and `list[8]` are `20` and `10`, respectively. Clearly, `list[3]` is larger than `list[7]` and `list[8]`.

In heapsort, elements at position k, $2k + 1$, and $2k + 2$, if they exist, are accessed frequently. Therefore, to facilitate the discussion of heapsort, we typically view data in the form of a complete binary tree as described next. For example, the data given in Figure 10-41 can be viewed in a complete binary tree, as shown in Figure 10-42.

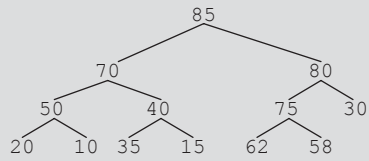


FIGURE 10-42 Complete binary tree corresponding to the list in Figure 10-41

In Figure 10-42, the first element of the list, which is `85`, is the *root* node of the tree. The second element of the list, which is `70`, is the *left child* of the root node; the third element of the list, which is `80`, is the *right child* of the root node. Thus, in general, for the node k, which is the $k - 1$th element of the list, its left child is the $2k$th (if it exists) element of the list, which is at position $2k - 1$ in the list, and the right child is the $2k + 1$st (if it exists) element of the list, which is at position $2k$ in the list. Note that Figure 10-42 clearly shows that the list in Figure 10-41 is in a heap. Also note that in Figure 10-42, the elements `20`, `10`, `35`, `15`, `62`, `58`, and `30` are called *leaves* as they have no children.

As remarked earlier, to demonstrate the heapsort algorithm, we will draw the complete binary tree corresponding to a list. Note that even though we will draw a complete binary

tree to illustrate heapsort, the data gets manipulated in an array. We now describe heapsort.

The first step in heapsort is to convert the list into a heap, called `buildHeap`. After we convert the array into a heap, the sorting phase begins.

Build Heap

This section describes the build heap algorithm.

The general algorithm is as follows: Suppose `length` denotes the length of the list. Let `index = length / 2 - 1`. Then `list[index]` is the last element in the list which is not a leaf; that is, this element has at least one child. Thus, elements `list[index + 1] ...list[length - 1]` are leaves.

First, we convert the subtree with the root node `list[index]` into a heap. Note that this subtree has at most three nodes. We then convert the subtree with the root node `list[index - 1]` into a heap, and so on.

To convert a subtree into a heap, we perform the following steps: Suppose that `list[a]` is the root node of the subtree, `list[b]` is the left child, and `list[c]`, if it exists, is the right child of `list[a]`.

Compare `list[b]` with `list[c]` to determine the larger child. If `list[c]` does not exist, then `list[b]` is the larger child. Suppose that `largerIndex` indicates the larger child. (Notice that, `largerIndex` is either `b` or `c`.)

Compare `list[a]` with `list[largerIndex]`. If `list[a] < list[largerIndex]`, then swap `list[a]` with `list[largerIndex]`; otherwise, the subtree with root node `list[a]` is already in a heap.

Suppose that `list[a] < list[largerIndex]` and we swap the elements `list[a]` with `list[largerIndex]`. After making this swap, the subtree with root node `list[largerIndex]` might not be in a heap. If this is the case, then we repeat Steps 1 and 2 at the subtree with root node `list[largerIndex]` and continue this process until either the heap in the subtrees is restored or we arrive at an empty subtree. This step is implemented using a loop, which will be described when we write the algorithm.

Consider the list in Figure 10-43. Let us call this `list`.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
list	15	60	72	70	56	32	62	92	45	30	65

FIGURE 10-43 Array `list`

Figure 10-44 shows the complete binary tree corresponding to the list in Figure 10-43.

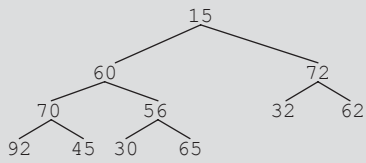


FIGURE 10-44 Complete binary tree corresponding to the list in Figure 10-43

To facilitate this discussion, when we say node `56`, we mean the node with info `56`.

This list has `11` elements, so the length of the list is `11`. To convert the array into a heap, we start at the list element `n/2 - 1 = 11/2 - 1 = 5 - 1 = 4`, which is the fifth element of the list.

Now `list[4] = 56`. The children of `list[4]` are `list[4 * 2 + 1]` and `list[4 * 2 + 2]`, that is, `list[9]` and `list[10]`. In the previous list, both `list[9]` and `list[10]` exist. To convert the tree with root node `list[4]`, we perform the previous hree steps:

1. Find the larger of `list[9]` and `list[10]`, that is, the largest child of `list[4]`. In this case, `list[10]` is larger than `list[9]`.
2. Compare the larger child with the parent node. If the larger child is larger than the parent, swap the larger child with the parent. Because `list[4] < list[10]`, we swap `list[4]` with `list[10]`.
3. Because `list[10]` does not have a subtree, Step 3 does not execute.

Figure 10-45(a) shows the resulting binary tree.

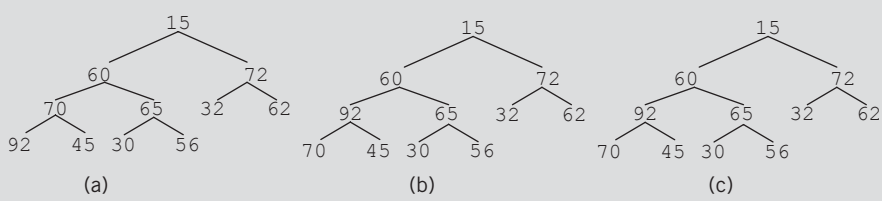


FIGURE 10-45 Binary tree while building heaps at `list[4]`, `list[3]`, and `list[2]`

Next, we consider the subtree with root node `list[3]`, that is, `70` and repeat the three steps given earlier, to obtain the complete binary tree as given in Figure 10-45(b). (Notice that Step 3 does not execute here either.)

Now we consider the subtree with the root node `list[2]`, that is, `72`, and apply the three steps given earlier. Figure 10-45(c) shows the resulting binary tree. (Note that in this case, because the parent is larger than both children, this subtree is already in a heap.)

Next, we consider the subtree with the root node `list[1]`, that is, `60`, see 10-45(c). First, we apply Steps 1 and 2. Because `list[1] = 60 < list[3] = 92` (the larger child), we swap `list[1]` with `list[3]`, to obtain the tree as given in Figure 10-46(a).

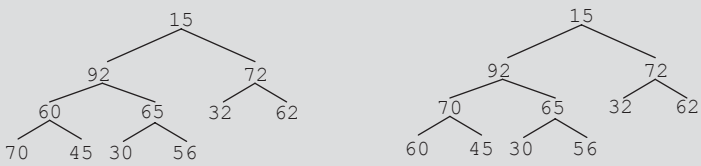


FIGURE 10-46 Binary tree while building heap at `list[1]`

However, after swapping `list[1]` with `list[3]`, the subtree with the root node `list[3]`, that is, `60`, is no longer a heap. Thus, we must restore the heap in this subtree. To do this, we apply Step 3 and find the larger child of `60` and swap it with `60`. We then obtain the binary tree as given in Figure 10-46(b).

Once again, the subtree with the root node `list[1]`, that is, `92`, is in a heap (see Figure 10-46(b)).

Finally, we consider the tree with the root node `list[0]`, that is, `15`. We repeat the previous three steps to obtain the binary tree as given in Figure 10-47(a).

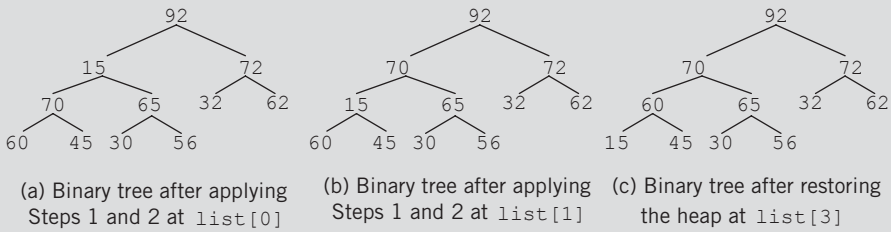


FIGURE 10-47 Binary tree while building heap at `list[0]`

We see that the subtree with the root node `list[1]`, that is, `15`, is no longer in a heap. So we must apply Step 3 to restore the heap in this subtree. (This requires us to repeat Steps 1 and 2 at the subtree with root node `list[1]`.) We swap `list[1]` with the larger child, which is `list[3]` (that is, `70`). We then get the binary tree of Figure 10-47(b).

The subtree with the root node `list[3] = 15` is not in a heap, and so we must restore the heap in this subtree. To do so, we apply Steps 1 and 2 at the subtree with root node `list[3]`. We swap `list[3]` with the larger child, which is `list[7]` (that is, `60`). Figure 10-47(c) shows the resulting binary tree.

The resulting binary tree in Figure 10-47(c) is in a heap, and so the list corresponding to this complete binary tree is in a heap.

Thus, in general, starting at the lowest level from right to left, we look at a subtree and convert the subtree into a heap as follows: If the root node of the subtree is smaller than the larger child, we swap the root node with the larger child. After swapping the root node with the larger child, we must restore the heap in the subtree whose root node was swapped.

Suppose `low` contains the index of the root node of the subtree and `high` contains the index of the last item in the list. The heap is to be restored in the subtree rooted at `list[low]`. The preceding discussion translates into the following C++ algorithm:

```
int largeIndex = 2 * low + 1;   //index of the left child

while (largeIndex <= high)
{
   if ( largeIndex < high)
       if (list[largeIndex] < list[largeIndex + 1])
          largeIndex = largeIndex + 1;  //index of the larger child
   if (list[low] > list[largeIndex])   //the subtree is already in
                                       //a heap
       break;
   else
   {
       swap(list[low], list[largeIndex]);   //Line swap**
       low = largeIndex;  //go to the subtree to further
                          //restore the heap
       largeIndex = 2 * low + 1;
   } //end else
}//end while
```

The `swap` statement at the line marked `Line swap**` swaps the parent with the larger child. Because a `swap` statement makes three item assignments to swap the contents of two variables, each time through the loop three item assignments are made. The `while` loop moves the parent node to a place in the tree so that the resulting subtree with the root node `list[low]` is in a heap. We can easily reduce the number of assignments each time through the loop from three to one by first storing the root node in a temporary location, say `temp`. Then each time through the loop, the larger child is compared with `temp`. If the larger child is larger than `temp`, we move the larger child to the root node of the subtree under consideration.

Next, we describe the function `heapify`, which restores the heap in a subtree by making one item assignment each time through the loop. The index of the root node

of the list and the index of the last element of the list are passed as parameters to this function.

```
template<class elemType>
void arrayListType<elemType>::heapify(int low, int high)
{
   int largeIndex;

   elemType temp = list[low]; //copy the root node of the subtree

   largeIndex = 2 * low + 1;  //index of the left child

   while (largeIndex <= high)
   {
       if (largeIndex < high)
          if (list[largeIndex] < list[largeIndex + 1])
             largeIndex = largeIndex + 1; //index of the largest
                                          //child

       if (temp > list[largeIndex]) //subtree is already in a heap
          break;
       else
       {
          list[low] = list[largeIndex]; //move the larger child
                                        //to the root
          low = largeIndex; //go to the subtree to restore the heap
          largeIndex = 2 * low + 1;
       }
   }//end while


   list[low] = temp; //insert temp into the tree, that is, list

} //end heapify
```

Next, we use the function `heapify` to implement the `buildHeap` function to convert the list into a heap.

```
template <class elemType>
void arrayListType<elemType>::buildHeap()
{
   for (int index = length / 2 - 1; index >= 0; index--)
       heapify(index, length - 1);
}
```

We now describe heapsort.

Suppose the list is in a heap. Consider the complete binary tree representing the list as given in Figure 10-48(a).

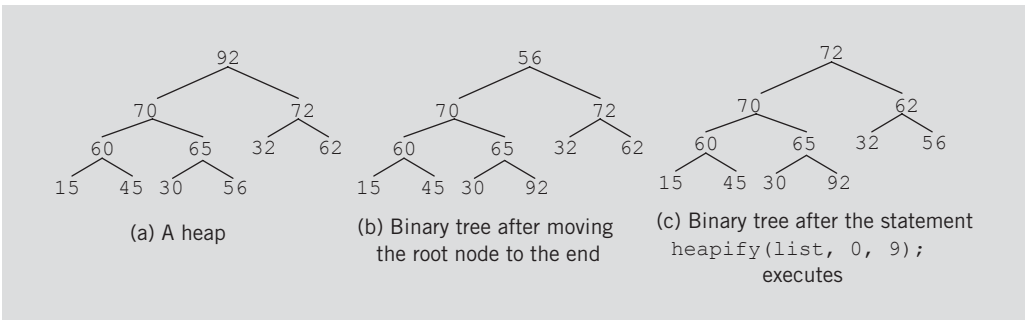


FIGURE 10-48 Heapsort

Because this is a heap, the root node is the largest element of the tree, that is, the largest element of the list. So it must be moved to the end of the list. We swap the root node of the tree, that is, the first element of the list, with the last node in the tree (which is the last element of the list). We then obtain the binary tree as shown in Figure 10-48(b).

Because the largest element is now in its proper place, we consider the remaining elements of the list, that is, elements `list[0]...list[9]`. The complete binary tree representing this list is no longer a heap, and so we must restore the heap in this portion of the complete binary tree. We use the function `heapify` to restore the heap. A call to this function is as follows:

```
heapify(list, 0, 9);
```

We thus obtain the binary tree as shown in Figure 10-48(c).

We repeat this process for the complete binary tree corresponding to the list elements `list[0]...list[9]`. We swap `list[0]` with `list[9]` and then restore the heap in the complete binary tree corresponding to the list elements `list[0]...list[8]`. We continue this process.

The following C++ function describes this algorithm:

```
template <class elemType>
void arrayListType<elemType>::heapSort()
{
    elemType temp;

    buildHeap();

    for (int lastOutOfOrder = length - 1; lastOutOfOrder >= 0;
                                          lastOutOfOrder--)
    {
        temp = list[lastOutOfOrder];
        list[lastOutOfOrder] = list[0];
        list[0] = temp;
        heapify(0, lastOutOfOrder - 1);
    }//end for
}//end heapSort
```

We leave as an exercise for you to write a program to test heapsort; see Programming Exercise 11 at the end of this chapter.

Analysis: Heapsort

Suppose that L is a list of n elements, where $n > 0$. In the worst case, the number of key comparisons in heapsort to sort L (the number of comparisons in `heapSort` and the number of comparisons in `buildHeap`) is $2n\log_2 n + O(n)$. Also, in the worst case, the number of item assignments in heapsort to sort L is $n\log_2 n + O(n)$. On average, the number of comparisons made by heapsort to sort L is of $O(n\log_2 n)$.

In the average case of quicksort, the number of key comparisons is $1.39n\log_2 n + O(n)$ and the number of swaps is $0.69n\log_2 n + O(n)$. Because each swap is three assignments, the number of item assignments in the average case of quicksort is at least $1.39n\log_2 n + O(n)$. It now follows that for the key comparisons, the average case of quicksort is somewhat better than the worst case of heapsort. On the other hand, for the item assignments, the average case of quicksort is somewhat poorer than the worst case of heapsort. However, the worst case of quicksort is of $O(n^2)$. Empirical studies have shown that heapsort usually takes twice as long as quicksort, but avoids the slight possibility of poor performance.

Priority Queues (Revisited)

Chapter 8 introduced priority queues. Recall that in a priority queue, customers or jobs with higher priorities are pushed to the front of the queue. Chapter 8 stated that we would discuss the implementation of priority queues after describing heapsort. For simplicity, we assume that the priority of the queue elements is assigned using the relational operators.

In a heap, the largest element of the list is always the first element of the list. After removing the largest element of the list, the function `heapify` restores the heap in the list. To ensure that the largest element of the priority queue is always the first element of the queue, we can implement priority queues as heaps. We can write algorithms similar to the ones used in the function `heapify` to insert an element in the priority queue (`addQueue` operation), and remove an element from the queue (`deleteQueue` operation). The next two sections describe these algorithms.

INSERT AN ELEMENT IN THE PRIORITY QUEUE

Assuming the priority queue is implemented as a heap, we perform the following steps:

1. Insert the new element in the first available position in the list. (This ensures that the array holding the list is a complete binary tree.)
2. After inserting the new element in the heap, the list might no longer be a heap. So to restore the heap:

```
while (the parent of the new entry is smaller than the new entry)
    swap the parent with the new entry.
```

Notice that restoring the heap might result in moving the new entry to the root node.

REMOVE AN ELEMENT FROM THE PRIORITY QUEUE

Assuming the priority queue is implemented as a heap, to remove the first element of the priority queue, we perform the following steps:

1. Copy the last element of the list into the first array position.
2. Reduce the length of the list by 1.
3. Restore the heap in the list.

The other operations for priority queues can be implemented in the same way as implemented for queues. We leave the implementation of the priority queues as an exercise for you; see Programming Exercise 12 at the end of this chapter.

PROGRAMMING EXAMPLE: Election Results

The presidential election for the student council of your local university is about to be held. The chair of the election committee wants to computerize the voting and has asked you to write a program to analyze the data and report the winner.

The university has four major divisions, and each division has several departments. For the election, the four divisions are labeled as region 1, region 2, region 3, and region 4. Each department in each division handles its own voting and reports the votes received by each candidate to the election committee. The voting is reported in the following form:

```
firstName lastName regionNumber numberOfVotes
```

The election committee wants the output in the following tabular form:

```
                    ----------Election Results----------

                                                 Votes
  Candidate Name    Region1  Region2  Region3  Region4   Total
------------------  -------  -------  -------  -------  ------
Sheila   Bower          23       70      133      267     493
Danny    Dillion        25       71      156       97     349
Lisa     Fisher        110      158        0        0     268
Greg     Goldy          75       34      134        0     243
Peter    Lamba         285       56        0       46     387
Mickey   Miller        112      141      156       67     476

Winner: Sheila Bower, Votes Received: 493

Total votes polled: 2216
```

The names of the candidates must be in alphabetical order in the output.

For this program, we assume that six candidates are seeking the student council's president post. This program can be enhanced to handle any number of candidates.

The data are provided in two files. One file, `candData.txt`, consists of the names of the candidates seeking the president's post. The names of the candidates in this file are in no particular order. In the second file, `voteData.txt`, each line consists of the voting results in the following form:

```
firstName lastName regionNumber numberOfVotes
```

Each line in the file `voteData.txt` consists of the candidate's name, the region number, and the number of votes received by the candidate in that region. There is one entry per line. For example, the input file containing the voting data looks like the following:

```
Greg Goldy 2 34
Mickey Miller 1 56
Lisa Fisher 2 56
.
.
.
```

The first line indicates that `Greg Goldy` received `34` votes from region `2`.

Input Two files: one containing the candidates' names and the other containing the voting data as described previously.

Output The election results in a tabular form, as described previously, and the winner.

PROBLEM ANALYSIS AND ALGORITHM DESIGN

From the output, it is clear that the program must organize the voting data by region and calculate the total votes both received by each candidate and polled for the election. Furthermore, the names of the candidates must appear in alphabetical order.

The main component of this program is a candidate. Therefore, first we design a `class candidateType` to implement a candidate object. Every candidate has a name and receives votes. Because there are four regions, we can use an array of four components. In Example 1-12 (Chapter 1), we designed the `class personType` to implement the name of a person. Recall that an object of the type `personType` can store the first name and the last name. Now that we have discussed operator overloading (see Chapter 2), we can redesign the `class personType` and define the relational operators so that the names of two people can be compared. We can also overload the assignment operator for easy assignment, and use the stream insertion and extraction operators for input/output. Because every candidate is a person, we derive the `class candidateType` from the `class personType`.

personType

The **class personType** implements the first name and last name of a person. Therefore, the **class personType** has two data members: a data member, **firstName**, to store the first name and a data member, **lastName**, to store the last name. We declare these as **protected** so that the definition of the **class personType** can be easily extended to accommodate the requirements of a specific application needed to implement a person's name. The definition of the **class personType** is given next:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement a person's
// name.
//*************************************************************

#include <iostream>
#include <string>

using namespace std;

class personType
{
        //Overload the stream insertion and extraction operators.
    friend istream& operator>>(istream&, personType&);
    friend ostream& operator<<(ostream&, const personType&);

public:
    const personType& operator=(const personType&);
      //Overload the assignment operator.

    void setName(string first, string last);
      //Function to set firstName and lastName according to
      //the parameters.
      //Postcondition: firstName = first; lastName = last

    string getFirstName() const;
      //Function to return the first name.
      //Postcondition: The value of firstName is returned.

    string getLastName() const;
      //Function to return the last name.
      //Postcondition: The value of lastName is returned.

    personType(string first = "", string last = "");
      //constructor with parameters
      //Set firstName and lastName according to the parameters.
      //Postcondition: firstName = first; lastName = last

        //Overload the relational operators.
    bool operator==(const personType& right) const;
    bool operator!=(const personType& right) const;
    bool operator<=(const personType& right) const;
```

```
    bool operator<(const personType& right) const;
    bool operator>=(const personType& right) const;
    bool operator>(const personType& right) const;

protected:
    string firstName; //variable to store the first name
    string lastName;  //variable to store the last name
};
```

We only give the definitions of the functions to overload the operators == and >> and leave others as an exercise for you; see Programming Exercise 13 at the end of this chapter.

```
    //overload the operator ==
bool personType::operator==(const personType& right) const
{
   return (firstName == right.firstName
          && lastName == right.lastName);
}

    //overload the stream insertion operator
istream& operator>>(istream& isObject, personType& pName)
{
       isObject >> pName.firstName >> pName.lastName;

       return isObject;
}
```

candidateType

The main component of this program is the candidate, which is described and implemented in this section. Every candidate has a first and a last name, and receives votes. Because there are four regions, we declare an array of four components to keep track of the votes for each region. We also need a data member to store the total number of votes received by each candidate. Because every candidate is a person and we have designed a class to implement the first and last name, we derive the **class candidateType** from the **class personType**. Because the data members of the **class personType** are **protected**, these data members can be accessed directly in the **class candidateType**.

There are six candidates. Therefore, we declare a list of six candidates of type **candidateType**. This chapter discussed sorting algorithms and added these algorithms to the **class arrayListType**. In Chapter 9, we derived the **class orderedArrayList** from the class **arrayListType** and included the binary search algorithm. We will use this class to maintain the list of candidates. This list of candidates will be sorted and searched. Therefore, we must define (that is, overload) the assignment and relational operators for the **class candidateType** because these operators are used by the searching and sorting algorithms.

Data in the file containing the candidates' data consists of only the names of the candidates. Therefore, in addition to overloading the assignment operator so that the

value of one object can be assigned to another object, we also overload the assignment operator for the `class candidateType`, so that only the name (of the `personType`) of the candidate can be assigned to a candidate object. That is, we overload the assignment operator twice: once for objects of the type `candidateType`, and another for objects of the types `candidateType` and `personType`.

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement a candidate.
//*************************************************************

#include <string>
#include "personType.h"

using namespace std;

const int NO_OF_REGIONS = 4;

class candidateType: public personType
{
public:
    const candidateType& operator=(const candidateType&);
      //Overload the assignment operator for objects of the
      //type candidateType

    const candidateType& operator=(const personType&);
      //Overload the assignment operator for objects so that
      //the value of an object of type personType can be
      //assigned to an object of type candidateType

    void updateVotesByRegion(int region, int votes);
      //Function to update the votes of a candidate for a
      //particular region.
      //Postcondition: Votes for the region specified by the
      //    parameter are updated by adding the votes specified
      //    by the parameter votes.

    void setVotes(int region, int votes);
      //Function to set the votes of a candidate for a
      //particular region.
      //Postcondition: Votes for the region specified by the
      //    parameter are set to the votes specified by the
      //    parameter votes.

    void calculateTotalVotes();
      //Function to calculate the total votes received by a
      //candidate.
      //Postcondition: The votes in each region are added and
      //    assigned to totalVotes.
```

```
    int getTotalVotes() const;
      //Function to return the total votes received by a
      //candidate.
      //Postcondition: The value of totalVotes is returned.

    void printData() const;
      //Function to output the candidate's name, the votes
      //received in each region, and the total votes received.

    candidateType();
      //Default constructor.
      //Postcondition: Candidate's name is initialized to blanks,
      //    the number of votes in each region, and the total
      //    votes are initialized to 0.

        //Overload the relational operators.
    bool operator==(const candidateType& right) const;
    bool operator!=(const candidateType& right) const;
    bool operator<=(const candidateType& right) const;
    bool operator<(const candidateType& right) const;
    bool operator>=(const candidateType& right) const;
    bool operator>(const candidateType& right) const;

private:
    int votesByRegion[NO_OF_REGIONS];   //array to store the votes
                                        // received in each region
    int totalVotes; //variable to store the total votes
};
```

The definitions of the member functions of the `class candidateType` are given next.

To set the votes of a particular region, the region number and the number of votes are passed as parameters to the function `setVotes`. Because an array index starts at `0`, region `1` corresponds to the array component at position `0`, and so on. Therefore, to set the value of the correct array component, `1` is subtracted from the region. The definition of the function `setVotes` is as follows:

```
void candidateType::setVotes(int region, int votes)
{
    votesByRegion[region - 1] = votes;
}
```

To update the votes for a particular region, the region number and the number of votes for that region are passed as parameters. The votes are then added to the region's previous value. The definition of the function `updateVotesByRegion` is as follows:

```
void candidateType::updateVotesByRegion(int region, int votes)
{
    votesByRegion[region - 1] = votesByRegion[region - 1] + votes;
}
```

The definitions of the functions `calculateTotalVotes`, `getTotalVotes`, `printData`, the default constructor, and `getName` are given next:

```
void candidateType::calculateTotalVotes()
{
    totalVotes = 0;

    for (int i = 0; i < NO_OF_REGIONS; i++)
        totalVotes += votesByRegion[i];
}

int candidateType::getTotalVotes() const
{
    return totalVotes;
}

void candidateType::printData() const
{
    cout << left
         << setw(10) << firstName << " "
         << setw(10) << lastName << " ";

    cout << right;

    for (int i = 0; i < NO_OF_REGIONS; i++)
        cout << setw(7) << votesByRegion[i] << "  ";
    cout << setw(7) << totalVotes << endl;
}

candidateType::candidateType()
{
    for (int i = 0; i < NO_OF_REGIONS; i++)
        votesByRegion[i] = 0;

    totalVotes = 0;
}
```

To overload the relational operators for the `class candidateType`, the names of the candidates are compared. For example, two candidates are the same if they have the same name. The definitions of these functions are similar to the definitions of the functions to overload the relational operators for the `class personType`. We only give the definition of the function to overload the operator `==` and leave others as an exercise for you; see Programming Exercise 13.

```
bool candidateType::operator==(const candidateType& right) const
{
    return (firstName == right.firstName
           && lastName == right.lastName);
}
```

The definitions of the functions to overload the assignment operators for the `class candidateType` are also left as an exercise for you; see Programming Exercise 13.

MAIN PROGRAM

Now that the **class candidateType** has been designed, we focus on designing the main program.

Because there are six candidates, we create a list, **candidateList**, containing six components of the type **candidateType**. The first thing that the program should do is read each candidate's name from the file **candData.txt** into the list **candidateList**. Next, we sort **candidateList**.

The next step is to process the voting data from the file **voteData.txt**, which holds the voting data. After processing the voting data, the program should calculate the total votes received by each candidate and then print the data as shown previously. Thus, the general algorithm is as follows:

1. Read each candidate's name into **candidateList**.
2. Sort **candidateList**.
3. Process the voting data.
4. Calculate the total votes received by each candidate.
5. Print the results.

The following statement creates the object **candidateList** of type **orderedArrayListType**.

```
orderedArrayListType<candidateType> candidateList(NO_OF_CANDIDATES);
```

Figure 10-49 shows the object **candidateList**. Every component of the array **list** is an object of the type **candidateType**.

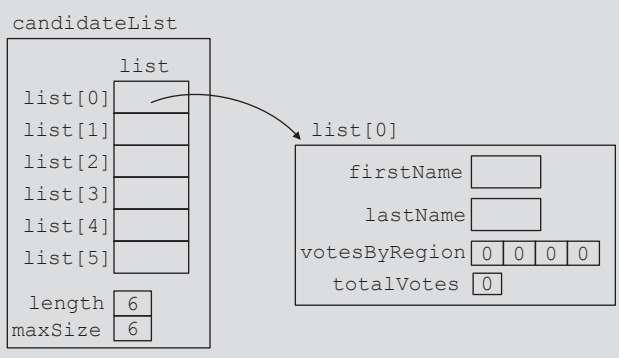


FIGURE 10-49 candidateList

In Figure 10-49, the array **votesByRegion** and the variable **totalVotes** are initialized to **0** by the default constructor of the **class candidateType**. To save

space, whenever needed, we will draw the object `candidateList` as shown in Figure 10-50.

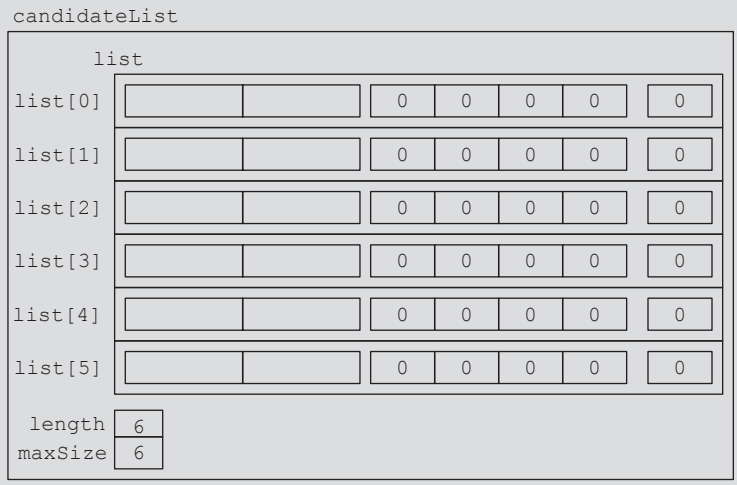


FIGURE 10-50 Object `candidateList`

`fillNames`

The first thing that the program must do is to read the candidates' names into `candidateList`. Therefore, we write a function to accomplish this task. The file `candData.txt` is opened in the function `main`. The name of the input file and `candidateList` are therefore passed as parameters to the function `fillNames`. Because the data member `list` of the object `candidateList` is a `protected` data member, it cannot be accessed directly. We, therefore, create an object, `temp`, of the type `candidateType`, to store the candidates' names, and use the function `insertAt` (of `list`) to store each candidate's name in the object `candidateList`. The definition of the function `fillNames` is as follows:

```
void fillNames(ifstream& inFile,
               orderedArrayListType<candidateType>& cList)
{
    string firstN;
    string lastN;

    candidateType temp;
```

```
    for (int i = 0; i < NO_OF_CANDIDATES; i++)
    {
        inFile >> firstN >> lastN;
        temp.setName(firstN, lastN);
        cList.insertAt(i, temp);
    }
}
```

After a call to the function **fillNames**, Figure 10-51 shows the object **candidateList**.

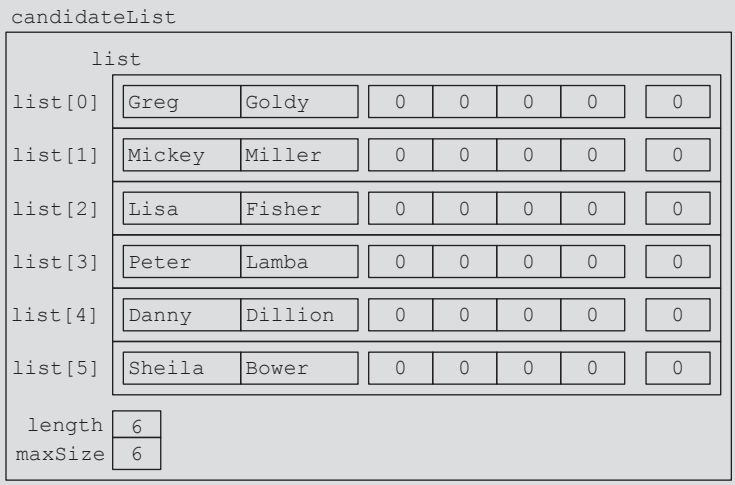


FIGURE 10-51 Object `candidateList` after a call to the function `fillNames`

Sort Names

After reading the candidates' names, next we sort the array **list** of the object **candidateList** using any of the (array-based) sorting algorithms discussed in this chapter. Because **candidateList** is an object of the type **orderedArrayListType**, all sorting algorithms discussed in this chapter are available to it. For illustration purposes, we use a selection sort. The following statement accomplishes this task:

```
candidateList.selectionSort();
```

After this statement executes, **candidateList** is as shown in Figure 10-52.

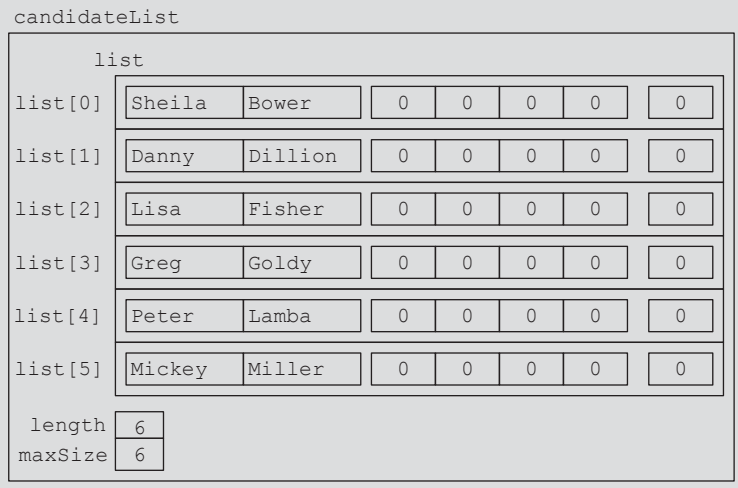


FIGURE 10-52 Object `candidateList` after the statement `candidateList.selectionSort();` executes

Process Voting Data

We now discuss how to process the voting data. Each entry in the file **`voteData.txt`** is of the form

```
firstName lastName regionNumber numberOfVotes
```

After reading an entry from the file **`voteData.txt`**, we locate the row in the array **`list`** (of the object **`candidateList`**) corresponding to the specific candidate, and update the entry specified by **`regionNumber`**.

The component **`votesByRegion`** is a **`private`** data member of each component of the array **`list`**. Moreover, **`list`** is a **`private`** data member of **`candidateList`**. The only way that we can update the votes of a candidate is to make a copy of that candidate's record into a temporary object, update the object, and then copy the temporary object back into **`list`** by replacing the old value with the new value of the temporary object. We can use the member function **`retrieveAt`** to make a copy of the candidate whose votes need to be updated. After updating the temporary object, we can use the member function **`replaceAt`** to copy the temporary object back into the list. Suppose the next entry read is

```
Lisa Fisher 2 35
```

This entry says that **`Lisa Fisher`** received **`35`** votes from region **`2`**. Suppose that before processing this entry, **`candidateList`** is as shown in Figure 10-53.

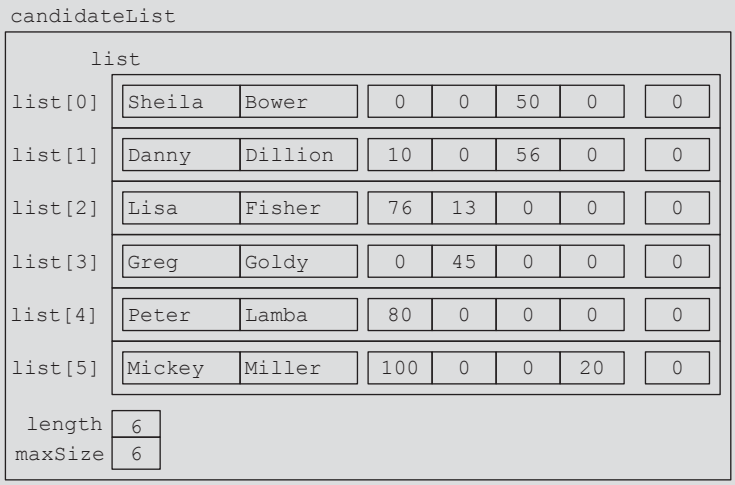


FIGURE 10-53 Object `candidateList` before processing the entry `Lisa Fisher 2 35`

We make a copy of the row corresponding to **`Lisa Fisher`** (see Figure 10-54).

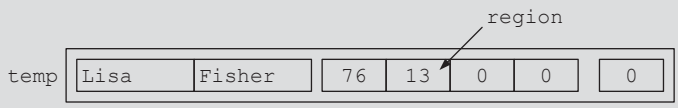


FIGURE 10-54 Object `temp`

Next, the following statement updates the voting data for region `2`. (Here **`region = 2`** and **`votes = 35`**.)

```
temp.updateVotesByRegion(region, votes);
```

After this statement executes, the object **`temp`** is as shown in Figure 10-55.

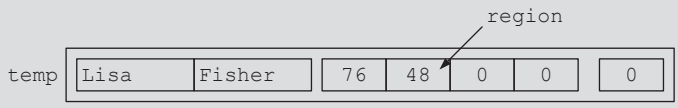


FIGURE 10-55 Object `temp` after the statement `temp.updateVotesByRegion(region, votes);` executes

Now we copy the object **temp** into **list** (see Figure 10-56).

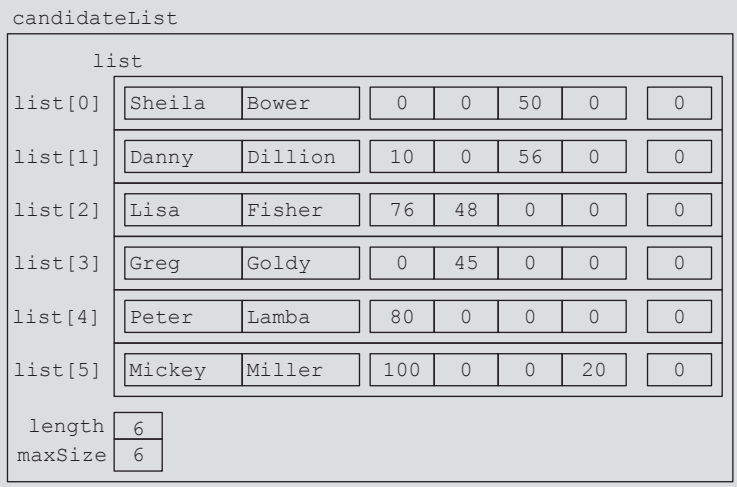


FIGURE 10-56 `candidateList` after copying `temp`

Because the member **list** of **candidateList** is sorted, we can use the binary search algorithm to find the row position in **list** corresponding to the candidate whose votes need to be updated. Also, the function **binarySearch** is a member of the **class orderedArrayListType**, so we can use this function to search the array **list**. We leave the definition of the function **processVotes** to process the voting data as an exercise for you; see Programming Exercise 13 at the end of this chapter.

Add Votes

After processing the voting data, the next step is to find the total votes received by each candidate. This is done by adding the votes received in each region. Now **votesByRegion** is a **private** data member of **candidateType** and **list** is a **protected** data member of **candidateList**. Therefore, to add the votes for each candidate, we use the **retrieveAt** function to make a temporary copy of each candidate's data, add the votes in the temporary object, and then copy the temporary object back into **candidateList**. The following function does this:

```
void addVotes(orderedArrayListType<candidateType>& cList)
{
    candidateType temp;

    for (int i = 0; i < NO_OF_CANDIDATES; i++)
```

```
    {
        cList.retrieveAt(i, temp);
        temp.calculateTotalVotes();
        cList.replaceAt(i, temp);
    }
}
```

Figure 10-57 shows `candidateList` after adding the votes for each candidate—that is, after a call to the function `addVotes`.

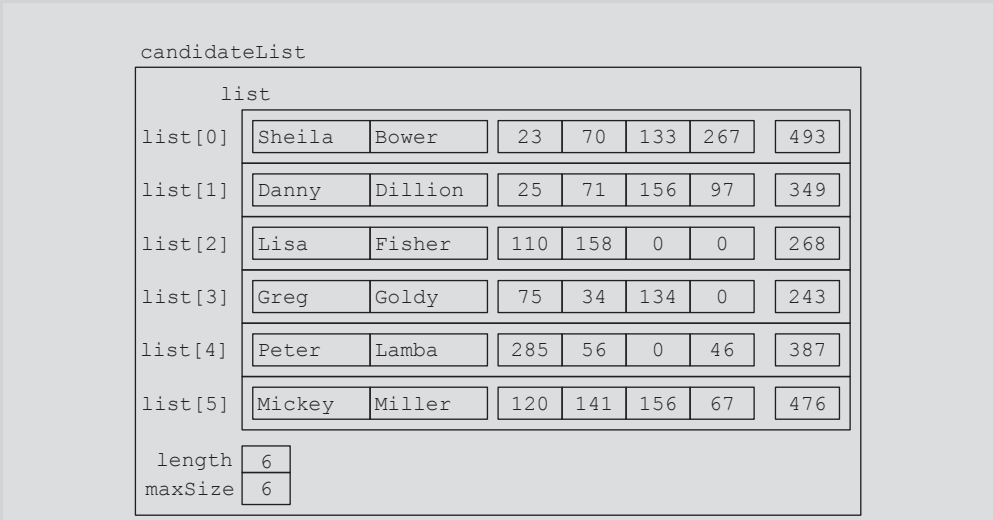


FIGURE 10-57 `candidateList` after a call to the function `addVotes`

10

Print Heading and Print Results

To complete the program, we include a function to print the heading, the first four lines of the output. The following function accomplishes this task:

```
void printHeading()
{
    cout << "      --------------------Election Results---------"
         << "-----------" << endl << endl;
    cout << "                                        Votes" << endl;
    cout << "    Candidate Name     Region1  Region2  Region3  "
         <<"Region4   Total" << endl;
    cout << "--------------------  -------  -------  "
         << "-------  -------  -----" << endl;
}
```

We now describe the function `printResults`, which prints the results. Suppose that the variable `sumVotes` holds the total votes polled for the election, the variable `largestVotes` holds the largest number of votes received by a candidate, and the variable `winLoc` holds the index of the winning candidate in the array `list`. Further suppose that `temp` is an object of the type `candidateType`. The algorithm for this function is as follows:

1. Initialize `sumVotes`, `largestVotes`, and `winLoc` to `0`.
2. For each candidate
   a. Retrieve the candidate's data into `temp`.
   b. Print the candidate's name and relevant data.
   c. Retrieve the total votes received by the candidate and update `sumVotes`.

```
if (largestVotes < temp.getTotalVotes())
{
    largestVotes = temp.getTotalVotes();
    winLoc = i; //this is the ith candidate
}
```

3. Output the final lines of the output.

We leave the definition of the function `printResults` to print the results as an exercise for you; see Programming Exercise 13 at the end of this chapter.

PROGRAM LISTING (MAIN PROGRAM)

```
//*************************************************************
// Author: D.S. Malik
//
// Program: Election Results
// Given candidates' voting this program determines the winner
// of the election. The program outputs the votes received by
// each candidate and the winner.
//*************************************************************

#include <iostream>
#include <string>
#include <fstream>
#include "candidateType.h"
#include "orderedArrayListType.h"

using namespace std;

const int NO_OF_CANDIDATES = 6;
```

```
void fillNames(ifstream& inFile,
               orderedArrayListType<candidateType>& cList);
void processVotes(ifstream& inFile,
                  orderedArrayListType<candidateType>& cList);
void addVotes(orderedArrayListType<candidateType>& cList);

void printHeading();

void printResults(orderedArrayListType<candidateType>& cList);

int main()
{
    orderedArrayListType<candidateType>
             candidateList(NO_OF_CANDIDATES);

    ifstream inFile;

    inFile.open("candData.txt");

    fillNames(inFile, candidateList);

    candidateList.selectionSort();

    inFile.close();

    inFile.open("voteData.txt");

    processVotes(inFile, candidateList);

    addVotes(candidateList);

    printHeading();

    printResults(candidateList);

    return 0;
}

//Place the definitions of the functions fillNames, addVotes,
//printHeading here. Also write and place the definitions
//of the functions processVotes and printResults here.
```

Sample Output (After you have written the definitions of the functions of the classes `personType` and `candidateType`, and the definitions of the function `processVotes` and `printResults`, and run your program, it should produce the following output; see Programming Exercise 13.)

```
      --------------------Election Results--------------------

                                                Votes
  Candidate Name     Region1   Region2   Region3   Region4    Total
------------------   -------   -------   -------   -------   ------
Sheila   Bower          23        70       133       267       493
Danny    Dillion        25        71       156        97       349
Lisa     Fisher        110       158         0         0       268
Greg     Goldy          75        34       134         0       243
Peter    Lamba         285        56         0        46       387
Mickey   Miller        112       141       156        67       476

Winner: Sheila Bower, Votes Received: 493

Total votes polled: 2216
```

Input Files

candData.txt

```
Greg Goldy
Mickey Miller
Lisa Fisher
Peter Lamba
Danny Dillion
Sheila Bower
```

voteData.txt

```
Greg Goldy 2 34
Mickey Miller 1 56
Lisa Fisher 2 56
Peter Lamba 1 78
Danny Dillion 4 29
Sheila Bower 4 78
Mickey Miller 2 63
Lisa Fisher 1 23
Peter Lamba 2 56
Danny Dillion 1 25
Sheila Bower 2 70
Peter Lamba 4 23
Danny Dillion 4 12
Greg Goldy 3 134
Sheila Bower 4 100
Mickey Miller 3 67
Lisa Fisher 2 67
Danny Dillion 3 67
Sheila Bower 1 23
Mickey Miller 1 56
Lisa Fisher 2 35
Sheila Bower 3 78
Peter Lamba 1 27
```

```
Danny Dillion 2 34
Greg Goldy 1 75
Peter Lamba 4 23
Sheila Bower 3 55
Mickey Miller 4 67
Peter Lamba 1 23
Danny Dillion 3 89
Mickey Miller 3 89
Peter Lamba 1 67
Danny Dillion 2 37
Sheila Bower 4 89
Mickey Miller 2 78
Lisa Fisher 1 87
Peter Lamba 1 90
Danny Dillion 4 56
```

QUICK REVIEW

1. Selection sort sorts a list by finding the smallest (or equivalently, the largest) element in the list, and moving it to the beginning (or the end) of the list.
2. For a list of length n, where $n > 0$, selection sort makes $(1/2)n(n-1)$ key comparisons and $3(n-1)$ item assignments.
3. For a list of length n, where $n > 0$, on average, insertion sort makes $(1/4)n^2 + O(n) = O(n^2)$ key comparisons and $(1/4)n^2 + O(n) = O(n^2)$ item assignments.
4. Empirical studies suggest that for large lists of size n, the number of moves in Shellsort is in the range of $n^{1.25}$ to $1.6n^{1.25}$.
5. Let L be a list of n distinct elements. Any sorting algorithm that sorts L by comparison of the keys only, in its worst case, makes at least $O(n\log_2 n)$ key comparisons.
6. Both quicksort and mergesort sort a list by partitioning the list.
7. To partition a list, quicksort first selects an item from the list, called the `pivot`. The algorithm then rearranges the elements so that the elements in one of the sublists are less than the `pivot`, and the elements in the second sublist are greater than or equal to the `pivot`.
8. In a quicksort, the sorting work is done in partitioning the list.
9. On average, the number of key comparisons in quicksort is $O(n\log_2 n)$. In the worst case, the number of key comparisons in quicksort is $O(n^2)$.
10. Mergesort partitions the list by dividing it in the middle.
11. In mergesort, the sorting work is done in merging the list.
12. The number of key comparisons in mergesort is $O(n\log_2 n)$.

13. A heap is a list in which each element contains a key, such that the key in the element at position k in the list is at least as large as the key in the element at position $2k + 1$ (if it exists) and $2k + 2$ (if it exists).
14. The first step in the heapsort algorithm is to convert the list into a heap, called `buildHeap`. After we convert the array into a heap, the sorting phase begins.
15. Suppose that L is a list of n elements, where $n > 0$. In the worst case, the number of key comparisons in heapsort to sort L is $2n\log_2 n + O(n)$. Also, in the worst case, the number of item assignments in heapsort to sort L is $n\log_2 n + O(n)$.

EXERCISES

1. Sort the following list using selection sort as discussed in this chapter. Show the list after each iteration of the outer **`for`** loop.

   26, 45, 17, 65, 33, 55, 12, 18
2. Sort the following list using selection sort as discussed in this chapter. Show the list after each iteration of the outer **`for`** loop.

   36, 55, 17, 35, 63, 85, 12, 48, 3, 66
3. Assume the following list of keys: 5, 18, 21, 10, 55, 20
   The first three keys are in order. To move 10 to its proper position using insertion sort as described in this chapter, exactly how many key comparisons are executed?
4. Assume the following list of keys: 7, 28, 31, 40, 5, 20

   The first four keys are in order. To move 5 to its proper position using insertion sort as described in this chapter, exactly how many key comparisons are executed?
5. Assume the following list of keys: 28, 18, 21, 10, 25, 30, 12, 71, 32, 58, 15
   This list is to be sorted using insertion sort as described in this chapter for array-based lists. Show the resulting list after six passes of the sorting phase—that is, after six iterations of the **`for`** loop.
6. Recall insertion sort for array-based lists as discussed in this chapter. Assume the following list of keys: 18, 8, 11, 9, 15, 20, 32, 61, 22, 48, 75, 83, 35, 3

   Exactly how many key comparisons are executed to sort this list using insertion sort?
7. Explain why the number of item movements in Shellsort is less than the number of item movements in insertion sort.
8. Consider the following list of keys: 80, 57, 65, 30, 45, 77, 27, 4, 90, 54, 45, 2, 63, 38, 81, 28, 62. Suppose that this list is to be sorted using Shellsort. Show the list during each increment, as in this chapter.

a. Use the increment sequence 1, 3, 5

b. Use the increment sequence 1, 4, 7.

9. Both mergesort and quicksort sort a list by partitioning the list. Explain how mergesort differs from quicksort in partitioning the list.

10. Assume the following list of keys: 16, 38, 54, 80, 22, 65, 55, 48, 64, 95, 5, 100, 58, 25, 36

    This list is to be sorted using quicksort as discussed in this chapter. Use `pivot` as the middle element of the list.

    a. Give the resulting list after one call to the `partition` procedure.

    b. Give the resulting list after two calls to the `partition` procedure.

11. Assume the following list of keys: 18, 40, 16, 82, 64, 67, 57, 50, 37, 47, 72, 14, 17, 27, 35

    This list is to be sorted using quicksort as discussed in this chapter. Use `pivot` as the median of the `first`, `last`, and `middle` elements of the list.

    a. What is the `pivot`?

    b. Give the resulting list after one call to the `partition` procedure.

12. Using the function `buildHeap` as given in this chapter, convert the following array into a heap. Show the final form of the array.

    47, 78, 81, 52, 50, 82, 58, 42, 65, 80, 92, 53, 63, 87, 95, 59, 34, 37, 7, 20

13. Suppose that the following list was created by the function `buildHeap` during the heap creation phase of heapsort.

    100, 85, 94, 47, 72, 82, 76, 30, 20, 60, 65, 50, 45, 17, 35, 14, 28, 5

    Show the resulting array after two passes of heapsort. (Use the `heapify` procedure as given in this chapter.) Exactly how many key comparisons are executed during the first pass?

14. Suppose that L is a list is of length n and it is sorted using insertion sort. If L is already sorted in the reverse order, show that the number of comparisons is $(1/2)(n^2 - n)$ and the number of item assignments is $(1/2)(n^2 + 3n) - 2$.

15. Suppose that L is a list is of length n and it is sorted using insertion sort. If L is already sorted, show that the number of comparisons is $(n - 1)$ and the number of item assignments is 0.

16. Write the definition of the `class arrayListType` that implements the sorting algorithms for array-based lists as discussed in this chapter.

17. Write the definition of the `class unorderedLinkedList` that implements the searching (described in Chapter 5) and sorting algorithms for linked lists as discussed in this chapter.

PROGRAMMING EXERCISES

1. Write and test a version of selection sort for linked lists.
2. Write a program to test insertion sort for array-based lists as given in this chapter.
3. Write a program to test insertion sort for linked lists as given in this chapter.
4. Write the definition of the function `intervalInsertionSort` described in Shellsort. Also write a program to test Shellsort given in this chapter.
5. Write a program to sort an array as follows.
   a. Use insertion sort to sort the array. Print the number of comparisons and the number of item movements.
   b. Use Shellsort to sort the array using the function `shellSort` given in this chapter. Print the number of comparisons and the number of item movements.
   c. Test your program on a list of 1,000 elements and on a list of 10,000 elements.
6. Write a program to test quicksort for array-based lists as given in this chapter.
7. Write and test a version of quicksort for linked lists.
8. (**C. A. R. Hoare**) Let L be a list of size n. Quicksort can be used to find the kth smallest item in L, where $0 \leq k \leq n - 1$, without completely sorting L. Write and implement a C++ function, `kThSmallestItem`, that uses a version of quicksort to determine the kth smallest item in L without completely sorting L.
9. Sort an array of 10,000 elements using quicksort as follows:
   a. Sort the array using `pivot` as the middle element of the array.
   b. Sort the array using `pivot` as the median of the first, last, and middle elements of the array.
   c. Sort the array using `pivot` as the middle element of the array. However, when the size of any sublist reduces to less than 20, sort the sublist using insertion sort.
   d. Sort the array using `pivot` as the median of the first, last, and middle elements of the array. When the size of any sublist reduces to less than 20, sort the sublist using insertion sort.
   e. Calculate and print the CPU time for each of the preceding four steps.
10. Write a program to test mergesort for linked lists as given in this chapter.
11. Write a program to test heapsort for array-based lists as given in this chapter.
12. a. Write the definition of the class template to define the priority queues, as discussed in this chapter as an abstract data type (ADT).

b. Write the definitions of the function templates to implement the operations of the priority queues as defined in part (a).

c. Write a program to test various operations of the priority queues.

13. a. Write the definitions of the functions of the `class personType`, of the Programming Example Election Results, not given in the programming example.

b. Write the definitions of the functions of the `class candidateType`, of the Programming Example Election Results, not given in the programming example.

c. Write the definitions of the function `processVotes` and `printResults`, of the Programming Example Election Results.

d. After completing parts a, b, and c, write a program to produce the output shown in the Sample Run of the Programming Example Election Results.

14. In the Programming Example Election Results, the `class candidateType` contains the function `calculateTotalVotes`. After processing the voting data, this function calculates the total number of votes received by a candidate. The function `updateVotesByRegion` (of the `class candidateType`) updates only the number of votes for a particular region. Modify the definition of this function so that it also updates the total number of votes received by the candidate. By doing so, the function `addVotes` in the main program is no longer needed. Modify and run your program with the modified definition of the function `updateVotesByRegion`.

15. In the Programming Example Election Results, the object `candidateList` of the type `orderedArrayListType` is declared to process the voting data. The operations of inserting a candidate's data and updating and retrieving the votes were somewhat complicated. To update the candidate's votes, we copied each candidate's data from `candidateList` into a temporary object of the type `candidateType`, updated the temporary object, and then replaced the candidate's data with the temporary object. This is because the data member's `list` is a `protected` member of `candidateList`, and each component of `list` is a `private` data member. In this exercise, you are to modify the Programming Example Election Results to simplify the accessing of a candidate's data as follows: Derive a `class candidateListType` from the `class orderedArrayListType`.

```
class candidateListType: public orderedArrayListType<candidateType>
{
public:
   candidateListType();
    //default constructor
   candidateListType(int size);
    //constructor
```

```
    void processVotes(string fName, string lName, int region,
                      int votes);
      //Function to update the number of votes for a
      //particular candidate for a particular region.
      //Postcondition: The name of the candidate, the region,
      //and the number of votes are passed as parameters.
    void addVotes();
      //Function to find the total number of votes received by
      //each candidate.
    void printResult() const;
      //Function to output the voting data.
};
```

Because the `class candidateListType` is derived from the `class orderedArrayListType`, and `list` is a `protected` data member of the `class orderedArrayListType` (inherited from the `class arrayListType`), `list` can be directly accessed by a member of the `class candidateListType`.

Write the definitions of the member functions of the `class candidateListType`. Rewrite and run your program using the `class candidateListType`.